MODULE 2:

SOLVING PROBLEMS BY SEARCHING

PROBLEM-SOLVING: PROBLEM-SOLVING AGENTS, EXAMPLE PROBLEMS, SEARCHING FOR SOLUTIONS UNINFORMED SEARCH STRATEGIES
CHAPTER 3 - 3.1, 3.2, 3.3, 3.4

Solving Problems by Searching

- *Reflex agent is simple*
  - base their actions on a direct mapping from states to actions
  - but cannot work well in environments where this mapping would be too large to store and would take too long to learn
- *Hence, goal-based agent is used*

Problem-solving agent

- **Problem-solving agent**
  - A kind of goal-based agent
  - It solves problem by finding sequences of actions that lead to desirable states (goals)
- *To solve a problem,*
  - *the first step is the goal formulation, based on the current situation*

Goal formulation

- Goals help organize behavior by limiting the objectives that the agent is trying to achieve.
- Goal formulation, based on the current situation and the agent's performance measure, is the first step in problem
- Reaching from initial state → goal state
  - *Actions are required*
- *Actions* are the operators
  - *causing transitions between world states*
  - ***Actions** should be abstract enough at a certain degree, instead of very detailed*
  - *E.g., **turn left** VS **turn left 30 degree**, etc.*

Problem formulation

- Problem formulation is the process of deciding what actions and states to consider, given a goal.
- The agent will not know which of its possible actions is best, because it does not know enough about the state
- E.g., driving Arad→ Bucharest
  - *in-between states and actions defined*
  - *States: Some places in Arad & Bucharest*
  - *Actions: Turn left, Turn right, go straight, accelerate & brake, etc.*

Search

- Because there are many ways to achieve the same goal
  - *Those ways are together expressed as a tree*
  - *Multiple options of unknown value at a point,*
    - the agent can examine different possible sequences of actions, and choose the best
  - *This process of looking for the best sequence is called search*
  - *The best sequence is then a list of actions, called solution*

Search-based agent

- The agent explores different possible sequences of actions (e.g., driving through different towns) to see which ones will lead to its goal of reaching Bucharest.
- The process of systematically exploring possible sequences of actions that lead from the current state (Arad) to the goal state (Bucharest) is called **search.**
- The agent operates using a **formulate-search-execute** strategy:
  - ***Formulate:*** *Define the goal (reach Bucharest) and the problem (choose the best path).*
  - ***Search:*** *Use a search algorithm to find a sequence of actions leading to the goal.*
  - ***Execute:*** *Carry out the actions suggested by the search to achieve the goal.*

Search algorithm

- Defined as
  - *taking a problem*
  - *and returns a solution*
- Once a solution is found
  - *the agent follows the solution*
  - *and carries out the list of actions – execution phase*
- Design of an agent
  - *"Formulate, search, execute"*
  - *The **Simple Problem-Solving Agent** follows cycle continuously, reacting to new percepts, updating its state, formulating new goals, and solving problems to reach its objectives.*

A simple problem-solving agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(p) returns an action
    inputs: p, a percept
    static: s, an action sequence, initially empty
            state, some description of the current world state
            g, a goal, initially null
            problem, a problem formulation
    state ← UPDATE-STATE(state, p)
    if s is empty then
        g ← FORMULATE-GOAL(state)
        problem ← FORMULATE-PROBLEM(state, g)
        s ← SEARCH(problem)
    action ← RECOMMENDATION(s, state)
    s ← REMAINDER(s, state)
    return action
```

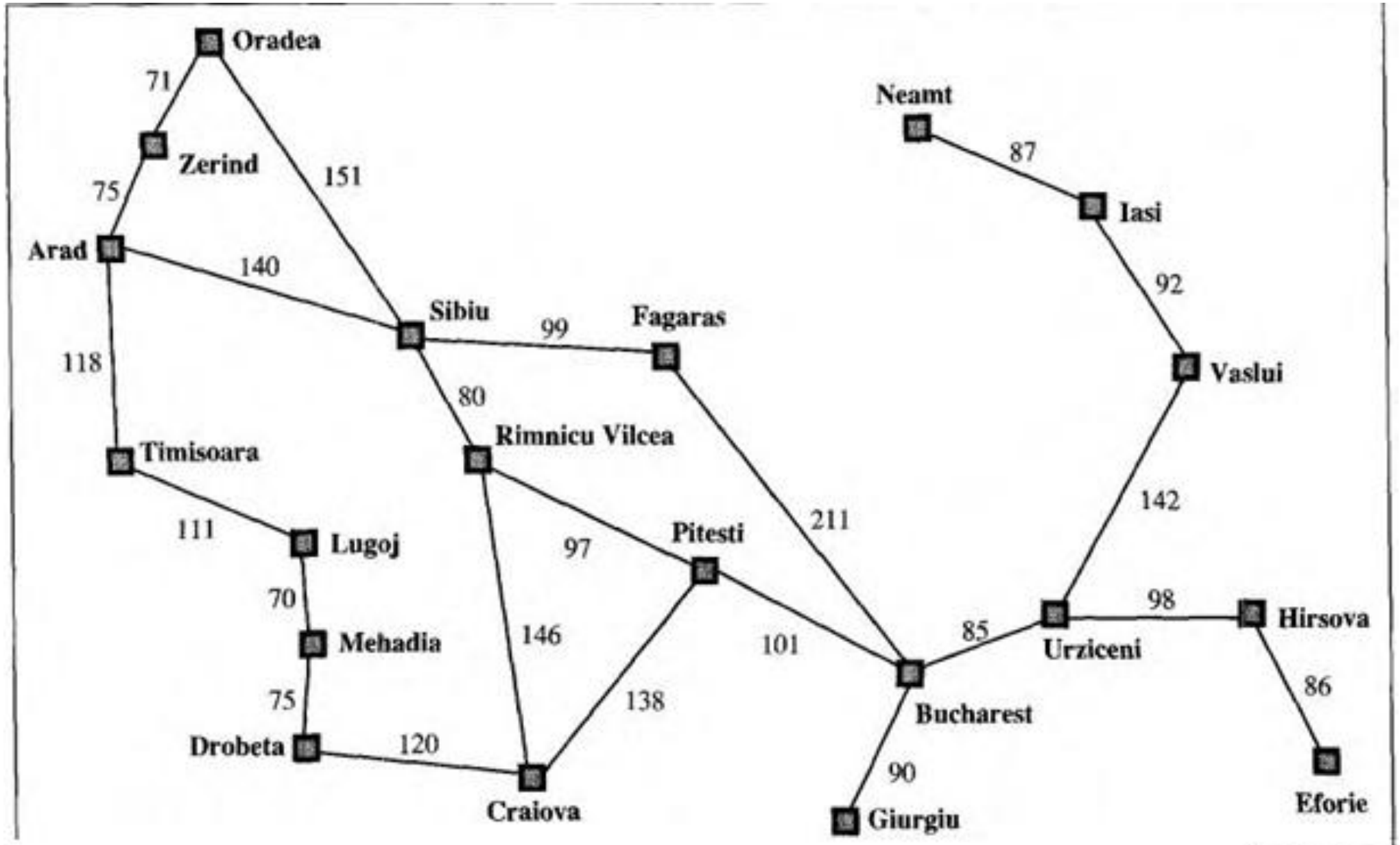


Figure 3.2 A simplified road map of part of Romania.

Well-defined problems and solutions

A problem is defined by 5 components:

- *Initial state*
- *Actions*
- *Transition model or (Successor functions)*
- *Goal Test.*
- *Path Cost.*

Well-defined problems and solutions

- A problem is defined by 4 components:
  - *The **initial state** that the agent starts in*
  - *The set of possible **actions***
  - ***Transition model:** description of what each action does.*

  *(**successor functions**): refer to any state reachable from given state by a single action*
  - *Initial state, actions and Transition model define the **state space***
    - the set of all states reachable from the initial state by any sequence of actions.
  - *A **path** in the state space:*
    - any sequence of states connected by a sequence of actions.

Well-defined problems and solutions

- The *goal test*
  - *Applied to the current state to test if the agent is in its goal*

  -Sometimes there is an explicit set of possible goal states. (example: in Bucharest).

  -Sometimes the goal is described by the properties instead of stating explicitly the set of states
  - *Example: Chess*
    - the agent wins if it can capture the KING of the opponent on next move (checkmate). No matter what the opponent does

Well-defined problems and solutions

- A ***path cost*** function,
  - *assigns a numeric cost to each path*
  - *= performance measure*
  - *denoted by* ***g*** *to distinguish the best path from others*
- Usually the path cost is
  - *the sum of the* ***step costs*** *of the individual actions (in the action list)*

Well-defined problems and solutions

- Together a problem is defined by
  - *Initial state*
  - *Actions*
  - *Successor function*
  - *Goal test*
  - *Path cost function*
- The ***solution*** of a problem is then
  - *a path from the initial state to a state satisfying the goal test*
- ***Optimal*** solution
  - *the solution with lowest path cost among all solutions*

Formulating problems

- Besides the four components for problem formulation
  - *anything else?*
- ***Abstraction***
  - *the process to take out the irrelevant information. Eg: turning on the radio, lookling out of the window*
  - *leave the most essential parts to the description of the states*

  (Remove detail from representation)
  - ***Conclusion:*** *Only the most important parts that are contributing to searching are used*

Problem-Solving Agents

- Agents whose task is to solve a particular problem (steps)
  - *goal formulation*
    - what is the goal state
    - what are important characteristics of the goal state
    - how does the agent know that it has reached the goal
    - are there several possible goal states
      - *are they equal or are some more preferable*
  - *problem formulation*
    - what are the possible states of the world relevant for solving the problem
    - what information is accessible to the agent
    - how can the agent progress from state to state

A Robot Vacuum Cleaner Navigating a House

- **Goal Formulation:**
  - ***What is the goal state?***
  - *The robot's goal is to clean the entire floor of a house.*
  - ***What are important characteristics of the goal state?***
  - *The goal is reached when every room is cleaned (i.e., no dirt is left on the floor).*
  - ***How does the agent know that it has reached the goal?***
  - *The robot detects that all sensors report a clean surface, with no dirt or debris in any room.*
  - ***Are there several possible goal states?***
  - *Yes, for example, the robot could prioritize cleaning certain rooms first or clean all rooms in a specific order.*
  - ***Are they equal or are some more preferable?***
    - Some paths may be more energy-efficient (e.g., minimizing the robot's battery usage or avoiding retracing steps), which would be a preferable goal state.

- Problem Formulation:
  - ***What are the possible states of the world relevant for solving the problem?***
  - *The current state of each room: clean or dirty. The position of obstacles like furniture that may block the robot's movement.*
  - ***What information is accessible to the agent?***
  - *The robot has sensors to detect dirt, walls, and obstacles. It also knows the layout of the house or can map it as it moves.*
  - ***How can the agent progress from state to state?***
  - *The robot moves from room to room, adjusting its path based on sensor inputs and updating its map of the environment, avoiding obstacles and ensuring all areas are cleaned.*

A Customer Support Chatbot Handling User Queries

- Problem Formulation:
  - ***What are the possible states of the world relevant for solving the problem?***
  - *The chatbot must account for the user's type of query, possible solutions from its database, and whether the issue requires human assistance.*
  - ***What information is accessible to the agent?***
  - *The chatbot has access to past chat records, the company's knowledge base, product information, and possibly some personal data about the user (e.g., purchase history).*
  - *How can the agent progress from state to state?The chatbot analyzes the user's query, identifies the nature of the problem (e.g., billing, technical issue), pulls relevant information, and responds to the user. If necessary, it escalates the problem to a human agent, moving from automated assistance to human intervention.*

From our Example

1. Formulate Goal

- Be In Bucharest

2. Formulate Problem

Problem : To Go from Arad to Bucharest

- States : Cities
- actions : Drive Between Cities

3. Find Solution

- Sequence of Cities : Arad to Sibiu to Rimnicu Vilcea to Pitesti to Bucharest.

Single-state problem formulation

A problem is defined by four items:

initial state e.g., "at Arad"

1. actions or successor function *S(x)* = set of action–state pairs
   - *e.g., S(Arad) = {<Arad → Zerind, Zerind>, … }*
2. goal test, can be
   - *explicit, e.g., x = "at Bucharest"*
   - *implicit, e.g., Checkmate(x)*
3. path cost (additive)
   - *e.g., sum of distances, number of actions executed, etc.*
   - *c(x,a,y) is the step cost, assumed to be ≥ 0*

- A solution is a sequence of actions leading from the initial state to a goal state

Example problems

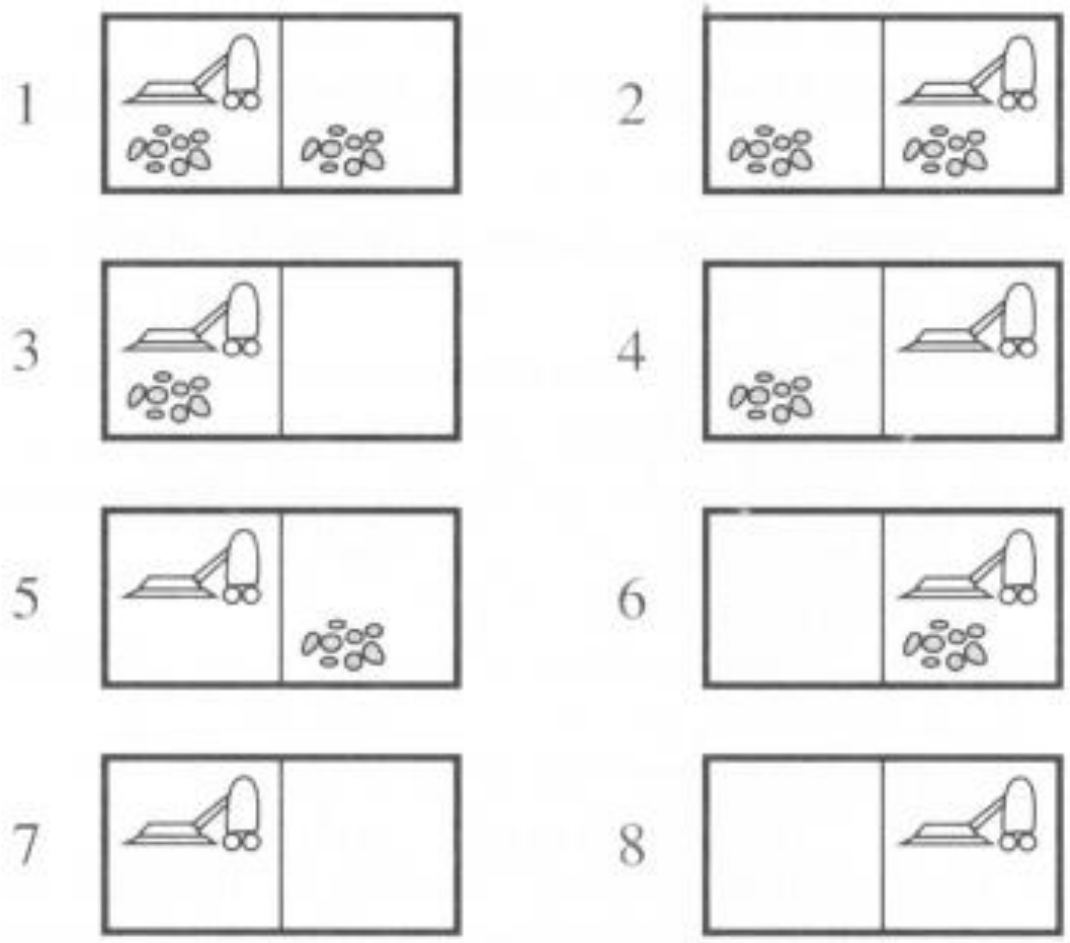
- *Toy problems*
  - *Those intended to illustrate or exercise various problem-solving methods*
  - *E.g., puzzle, chess, etc.*
- *Real-world problems*
  - *Tend to be more difficult and whose solutions people actually care about*
  - *E.g., Design, planning, etc.*

Vacuum world

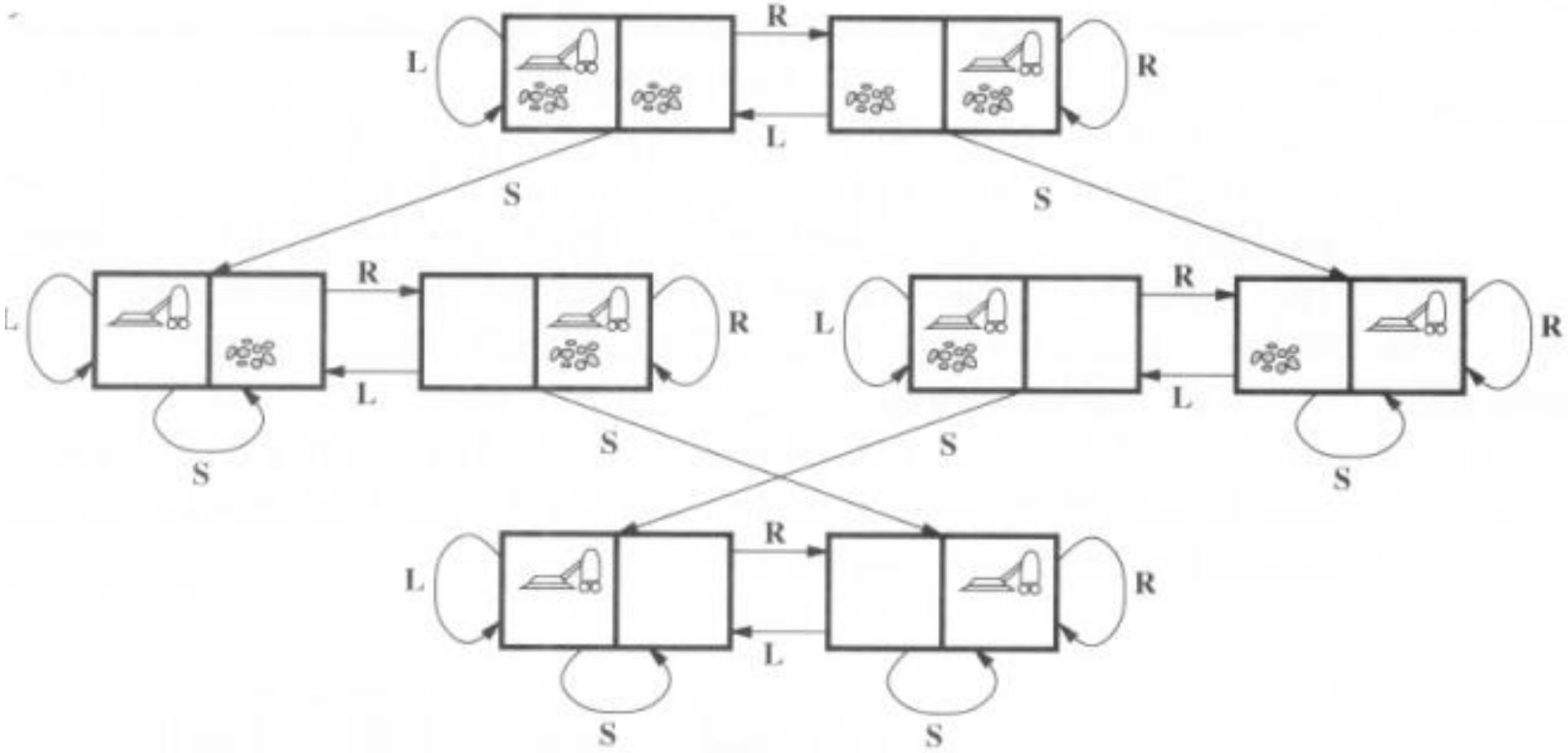
- **States**: The agent is in one of two locations, each of which might or might not contain dirt. Thus there are 2 x 2 = 8 possible world states.
- **Initial state**: Any state can be designated as the initial state.
- **Actions**: Each state has three actions: Left, Right, Suck.
- **Successor function**: This generates the legal states that result from trying the three actions (Left, Right, and Suck). The complete state space is shown in Figure 3.3.
- **Goal test**: This checks whether all the squares are clean.
- **Path cost**: Each step costs 1, so the path cost is the number of steps in the path.

Toy problems

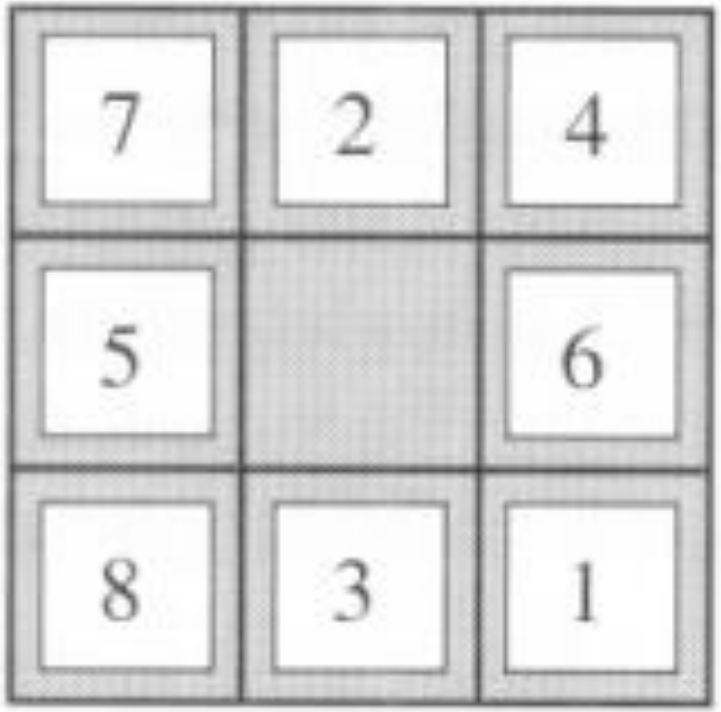
- Example: vacuum world



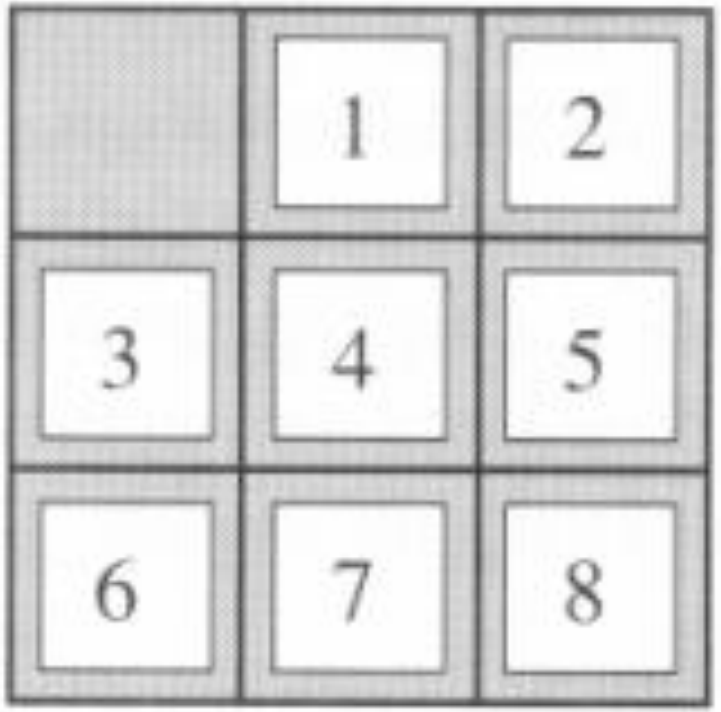
- Number of states: 8
- Initial state: Any
- Number of actions: 4
  - *left, right, suck, noOp*
- Goal: clean up all dirt
  - Goal states: {7, 8}
- Path Cost:
  - Each step costs 1


L
R
L
R
S
S
L
R
L
R
S
S
L
R
L
R
S
S
L
R
L
R
S
S

The 8-puzzle



Start State



Goal State

The 8-puzzle

- **States**: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state**: Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions**: The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.
- **Transition model**: Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test**: This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost**: Each step costs 1, so the path cost is the number of steps in the path.

The 8-puzzle

- The 8-puzzle has 9!/2 = 181, 440 reachable states and is easily solved.
- The 15-puzzle (on a 4×4 board) has around 1.3 trillion states, and random instances can be solved optimally in a few milliseconds by the best search algorithms.
- The 24-puzzle (on a 5 × 5 board) has around 1.55* pow(10)25 states, and random instances take several hours to solve optimally.

The 8-queens

- The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other.
- There are two ways to formulate the problem
  - *incremental formulation*
  - *A complete-state formulation*
- All of them have the common followings:
  - *Goal test: 8 queens on board, not attacking to each other*

The 8-queens

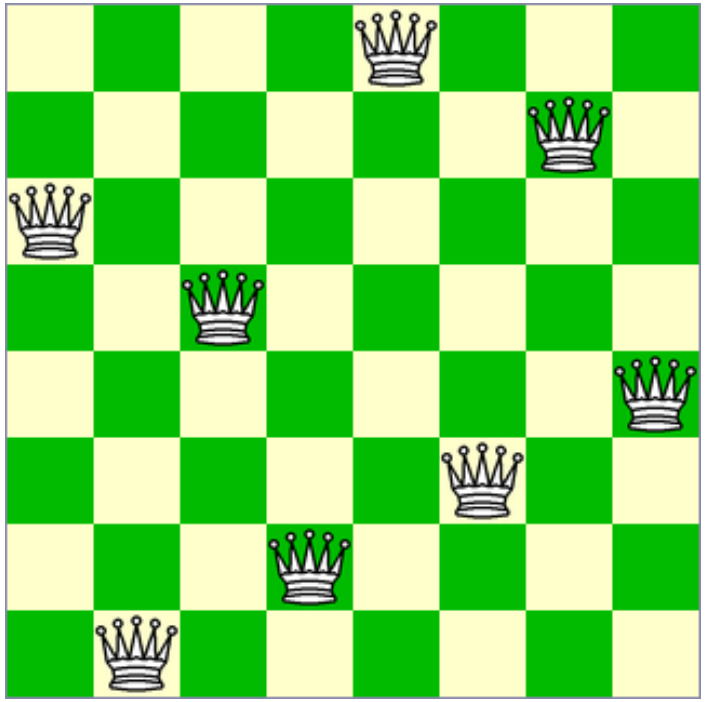
- (1) Incremental formulation
  - *involves operators that augment the state description starting from an empty state*
  - **States**: Any arrangement of 0 to 8 queens on the board is a state.
  - **Initial state**: No queens on the board.
  - **Actions**: Add a queen to any empty square.
  - **Transition model**: Returns the board with a queen added to the specified square.
  - **Goal test**: 8 queens are on the board, none attacked.
- In this formulation, we have 64 . 63 . . .57 = pow(10)14possible sequences to investigate.

The 8-queens

- (2) Complete-state formulation
  - *starts with all 8 queens on the board*
  - *move the queens individually around*
  - *States:*
    - any arrangement of 8 queens, one per column in the leftmost columns
  - *Successor function: Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen*
  - *Operators: move an attacked queen to a row, not attacked by any other*
- This formulation reduces the 8-queens state space from 1.8 x pow(10)14 to just 2,057, and solutions are easy to find.

The 8-queens

- Conclusion:
  - *the right formulation makes a big difference to the size of the search space*



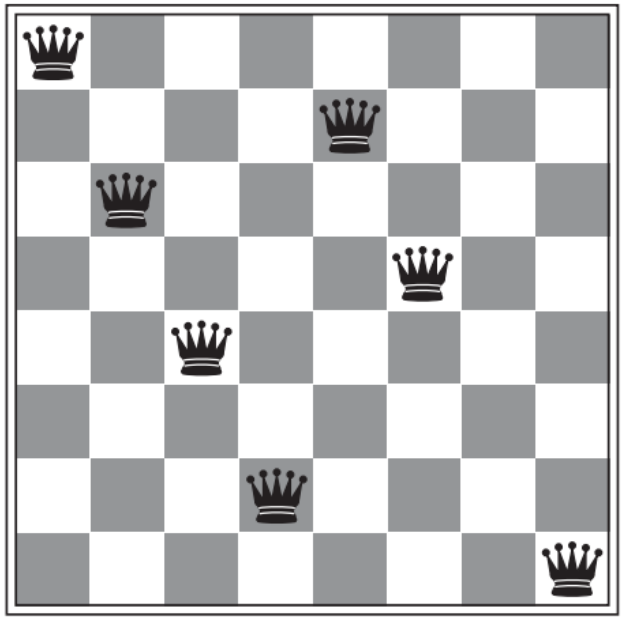


Figure 3.5 Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

Real-world problems

Route finding problem

- States: Each is represented by a location (e.g., an airport) and the current time.
- Initial state: This is specified by the problem.
- Successor function: This returns the states resulting from taking any scheduled flight (perhaps further specified by seat class and location), leaving later than the current time plus the within-airport transit time, from the current airport to another.
- Goal test: Are we at the destination by some prespecified time?
- Path cost: This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on.

Real-world problems

- Route-finding problems
- Touring problems
- Traveling Salesman problem
- VLSI layout problem
- Robot navigation
- Automatic assembly sequencing
- Internet searching

Touring problems

- Visit every city in at least once, starting and ending in Bucharest.”
- **Actions** correspond to trips between adjacent cities
- Each state must include not just the current location but also the *set of cities the agent has visited*.
- So the initial state would be In(Bucharest), Visited({Bucharest}), a typical intermediate state would be In(Vaslui), Visited({Bucharest, Urziceni, Vaslui})
- Goal test would check whether the agent is in Bucharest and all 20 cities have been visited.

Traveling Salesperson Problem (TSP)

- Given a set of cities and the distances between each pair of cities, the goal is to find the shortest possible route that visits every city exactly once and returns to the starting city.
- **NP-hard**: This means that there is no known algorithm that can solve all instances of the TSP quickly (in polynomial time) as the number of cities grows.
- In addition to planning trips for traveling salespersons, these algorithms have been used for tasks such as
  - *planning movements of automatic circuit-board drills*
  - *stocking machines on shop floors.*

Airline travel problems

- **States**: Each state obviously includes a location (e.g., an airport) and the current time. Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and their status as domestic or international, the state must record extra information about these "historical" aspects.
- **Initial state**: This is specified by the user's query.
- **Actions**: Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model**: The state resulting from taking a flight will have the flight's destination as the current location and the flight's arrival time as the current time.
- **Goal test**: Are we at the final destination specified by the user?
- **Path cost**: This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on.

VLSI LAYOUT

- **VLSI layout** problem requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield.
- The layout problem comes after the logical design phase and is usually split into two parts: ◦ **cell layout** and **channel routing**.
- **Cell layout:** the primitive components of the circuit are grouped into cells, each of which performs some recognized function.
  - *Each cell has a fixed footprint and requires a certain number of connections to each of the other cells.*
  - *The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells.*
- **Channel routing**
  - *finds a specific route for each wire through the gaps between the cells.*

Robot navigation

- **Robot navigation** is a generalization of the route-finding problem.
- Rather than following a discrete set of routes, a robot can move in a continuous space with
- an infinite set of possible actions and states.
- For a circular robot moving on a flat surface, the space is essentially two dimensional.
- When the robot has arms and legs or wheels that must also be controlled, the search space becomes many-dimensional.
- Advanced techniques are required just to make the search space finite.

Automatic assembly sequencing

- Aim is to find an order in which to assemble the parts of some object.
- If the wrong order is chosen, there will be no way to add some part later in the sequence without undoing some of the work already done.
- Checking a step in the sequence for feasibility is a difficult geometrical search problem closely related to robot navigation.
- Thus, the generation of legal actions is the expensive part of assembly sequencing.
- Any practical algorithm must avoid exploring all but a tiny fraction of the state space.
- **Protein design** is an automatic assembly problem in which the goal is to find a sequence of amino acids that will fold into a three-dimensional protein with the right properties to cure some disease.

3.3 Searching for solutions

3.3 Searching for solutions

- Finding out a solution is done by
  - *searching through the state space*
- All problems are transformed
  - *as a search tree*
  - *generated by the initial state and successor function*

Search tree

- **Initial state**
  - *The root of the search tree is a **search node***
- **Expanding**
  - *applying successor function to the current state*
  - *thereby generating a new set of states*
- **leaf nodes**
  - *the states having no successors*

  Fringe: Set of search nodes that have not been expanded yet.
- Refer to next figure

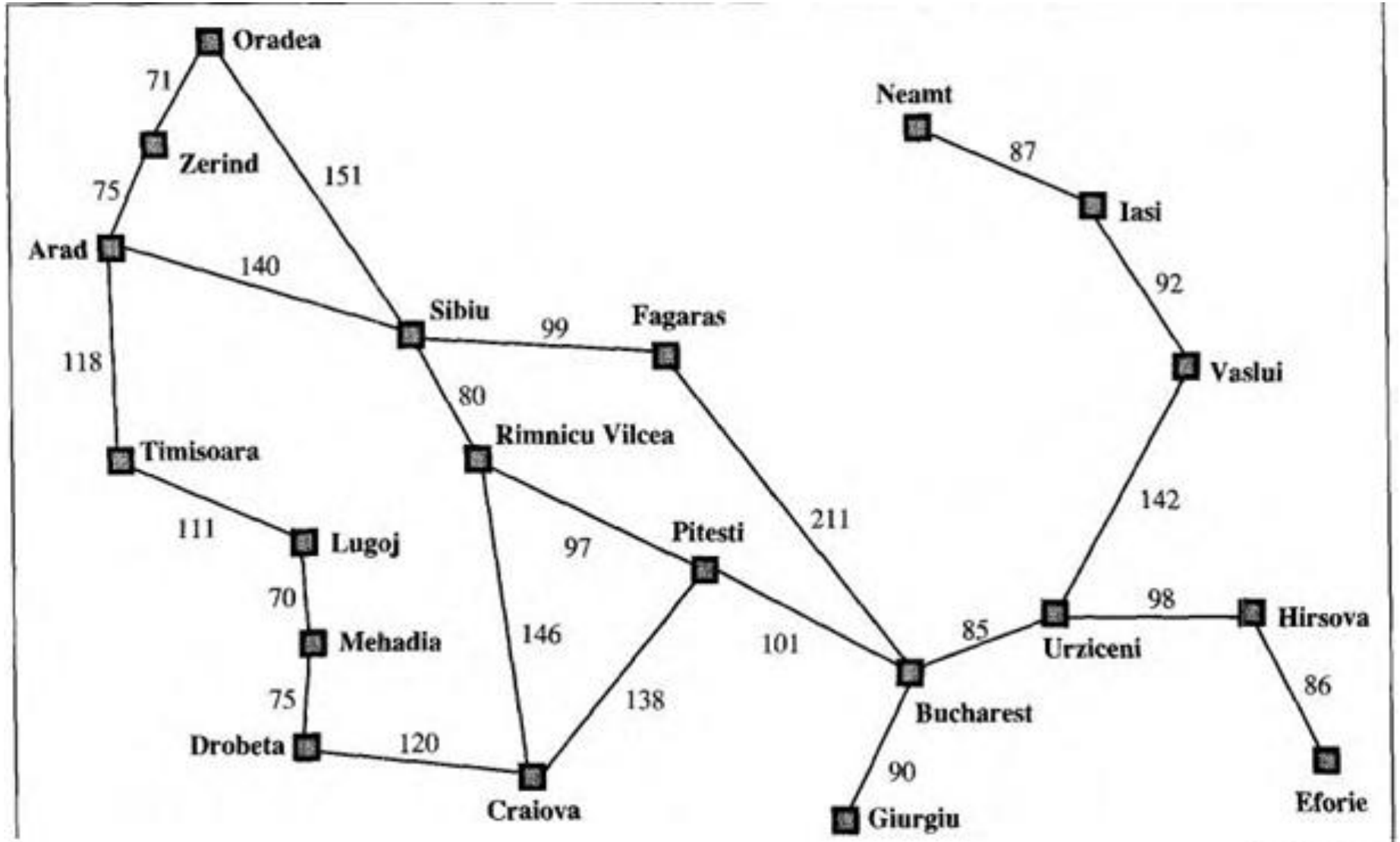
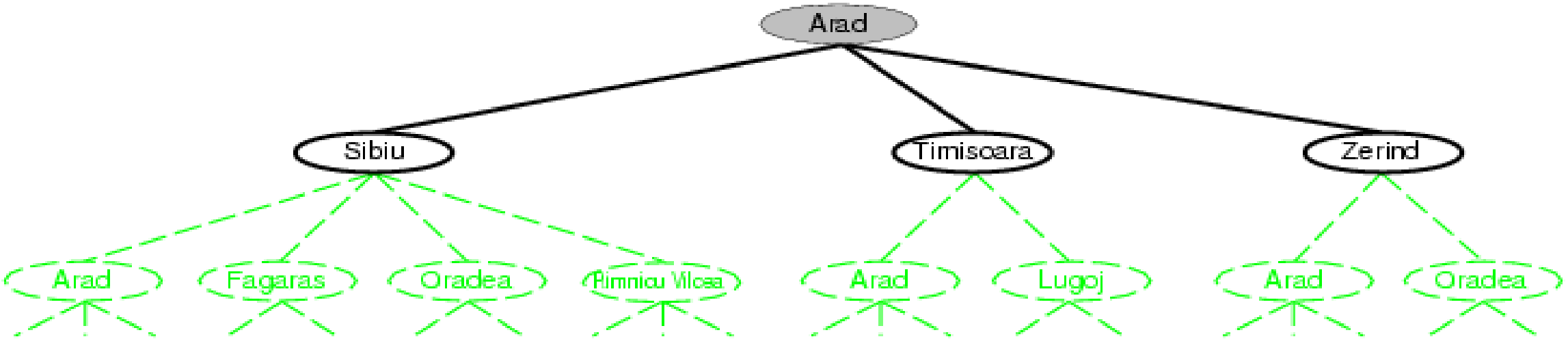
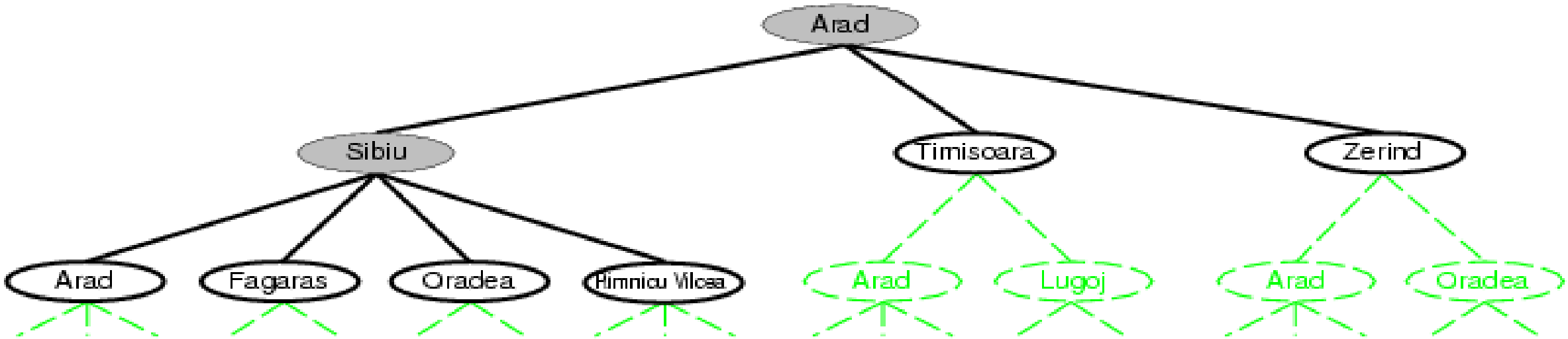


Figure 3.2 A simplified road map of part of Romania.

Tree search example



Tree search example



- The root node of the tree corresponds to the initial state, *In(Arad)*.
- **Expanding** the current state; applying each legal action to the current state, thereby **generating** a new set of states.
- We add three branches from the **parent node** *In(Arad)* leading to three new **child nodes**: *In(Sibiu), In(Timisoara),* and *In(Zerind)*.
- Now we must choose which of these three possibilities to consider further.
- The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.
- **Search strategy**: how they choose which state to expand next

- **Loopy path:** path from Arad to Sibiu and back to Arad again! We say that *In(Arad)* is a **repeated state** in the search tree, generated in this case by a **loopy path**
- Considering such loopy paths means that the complete search tree for Romania is *infinite* because there is no limit to how often one can traverse a loop
- loops can cause certain algorithms to fail, making otherwise solvable problems unsolvable.
- We can rely on more than intuition for this: because path costs are additive and step costs are nonnegative, a loopy path to any given state is never better than the same path with the loop removed.
- **Redundant paths:** exist whenever there is more than one way to get from one state to another eg, the paths
- Arad–Sibiu (140 km long) and Arad–Zerind–Oradea–Sibiu (297 km long).

TREE-SEARCH algorithm

- The data structure called the **explored set** (also known as the **closed list**), remembers every expanded node.
- Newly generated nodes that match previously generated nodes ones in the explored set or the frontier can be discarded instead of being added to the frontier.
- **Frontier is the reached but unexpanded states because we can expand only them**

```
function TREE-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

- **Initialize the Frontier**:
  - The **frontier** is initialized with the initial state of the problem.
- **Loop**:
  - The algorithm enters a loop where it will continue until either a solution is found or the frontier is empty.
- **Check if the Frontier is Empty**:
  - If there are no more nodes to explore (i.e., the frontier is empty), the algorithm returns **failure**. This means no solution exists for the problem.
- **Choose a Leaf Node**:
  - The algorithm selects a leaf node from the frontier and removes it. This node will be expanded next.
- **Goal Check**:
  - If the chosen node represents a goal state (i.e., it contains the solution to the problem), the algorithm returns the corresponding solution.
- **Expand the Chosen Node**:
- If the node is not a goal state, the algorithm expands it by generating its child nodes (i.e., resulting states from the possible actions).The child nodes are added to the frontier for future exploration.

function TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

---

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

Figure 3.7 An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

- **Tree-Search Algorithm**: This algorithm does not track previously visited nodes, which means it might revisit the same states multiple times.
- **Graph-Search Algorithm**: Graph-Search introduces an **explored set** to keep track of visited nodes and avoid revisiting them, which prevents looping and improves efficiency.
- When expanding a node, it only adds successors that are **not in the frontier or explored set**, ensuring that each state is explored once.

Search tree

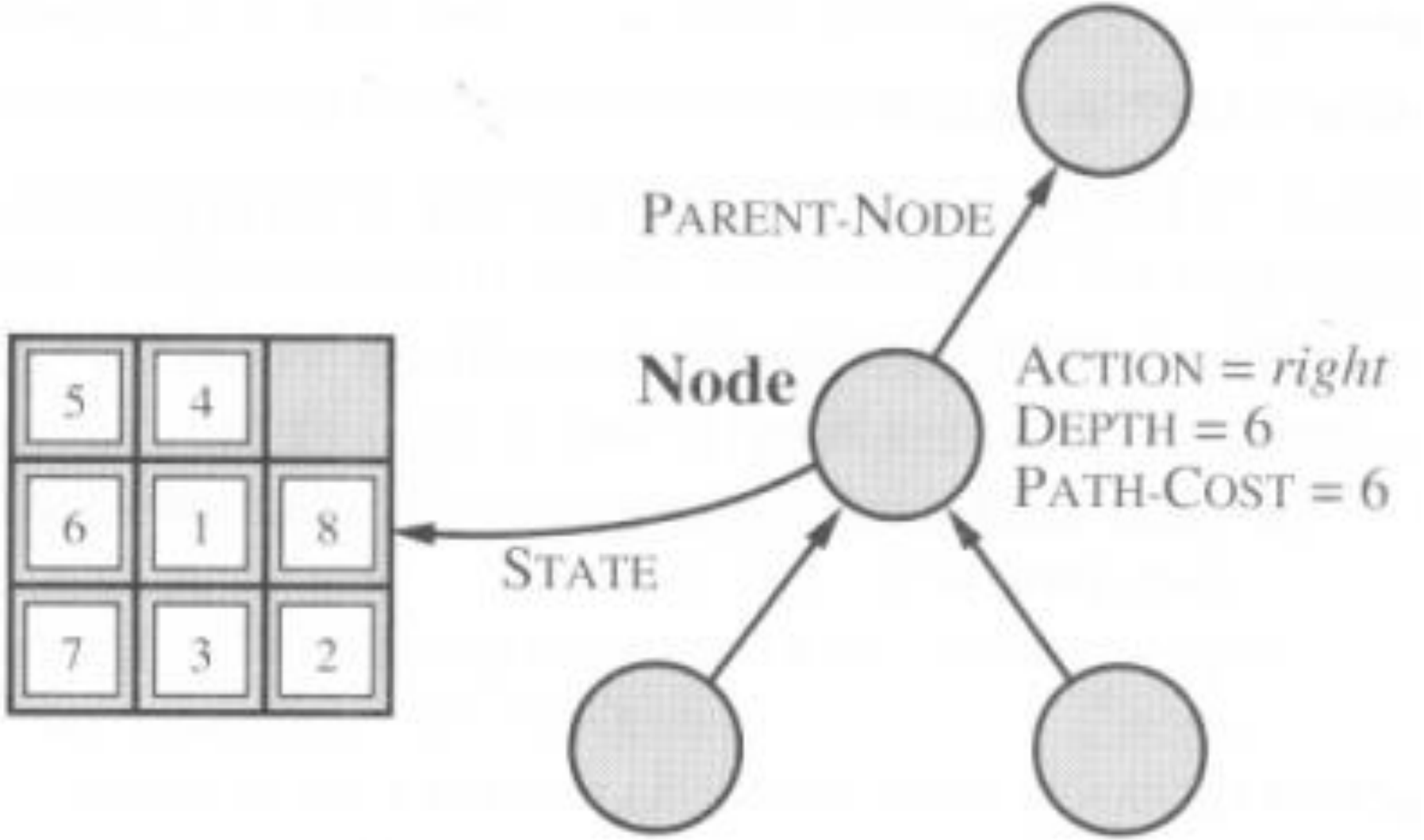
- The **essence** of searching
  - *in case the first choice is not correct*
  - *choosing one option and keep others for later inspection*
- Hence we have the **search strategy**
  - *which determines the choice of which state to expand*
  - *good choice → fewer work → faster*
- Important:
  - *state space ≠ search tree*

Search tree

- State space
  - *has unique states {A, B}*
  - *while a search tree may have cyclic paths: A-B-A-B-A-B- …*
- A good search strategy should avoid such paths

Search tree

- A node is having five components:
  - *STATE: which state it is in the state space*
  - *PARENT-NODE: from which node it is generated*
  - *ACTION: which action applied to its parent-node to generate it*
  - *PATH-COST: the cost, g(n), from initial state to the node n itself*
  - *DEPTH: number of steps along the path from the initial state*


PARENT-NODE
Node
ACTION = right
DEPTH = 6
PATH-COST = 6
STATE
5
4
6
1
8
7
3
2

Measuring problem-solving performance

- The evaluation of a search strategy
  - ***Completeness:***
    - is the strategy guaranteed to find a solution when there is one?
  - ***Optimality:***
    - does the strategy find the highest-quality solution when there are several different solutions?
  - ***Time complexity:***
    - how long does it take to find a solution?
  - ***Space complexity:***
    - how much memory is needed to perform the search?

Measuring problem-solving performance

- In AI, complexity is expressed in
  - ***b**, branching factor, maximum number of successors of any node*
  - ***d**, the depth of the shallowest goal node.*

  (depth of the least-cost solution)
  - ***m**, the maximum length of any path in the state space*
- Time and Space is measured in
  - *number of nodes generated during the search*
  - *maximum number of nodes stored in memory*

Measuring problem-solving performance

- For effectiveness of a search algorithm
  - *we can just consider the total cost*
  - ***The total cost*** = *path cost (g) of the solution found* **+** *search cost*
    - search cost = time necessary to find the solution
- Tradeoff:
  - *(long time, optimal solution with least g)*
  - *vs. (shorter time, solution with slightly larger path cost g)*

3.4 Uninformed search strategies

3.4 Uninformed search strategies

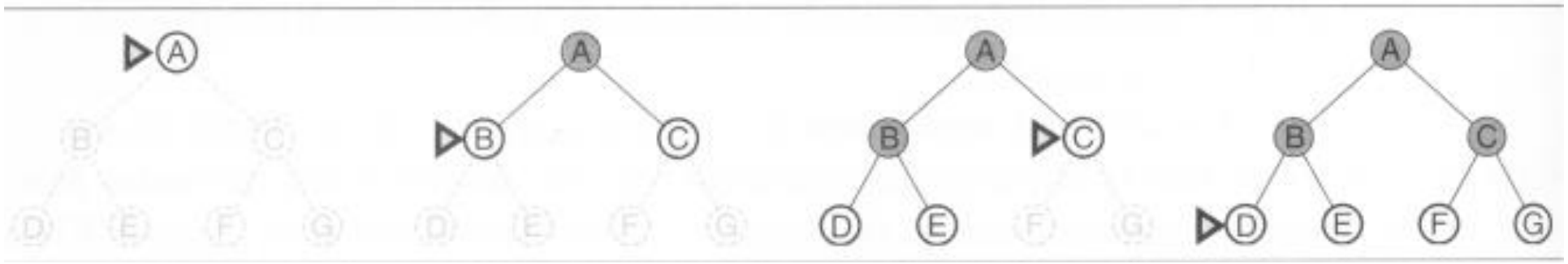
- **Uninformed search**
  - *no information about the number of steps*
  - *or the path cost from the current state to the goal*
  - *search the state space* ***blindly***
- **Informed search, or heuristic search**
  - *a cleverer strategy that searches toward the goal,*
  - *Strategies that know whether one goal is more promising than another.*
  - *based on the information from the current state so far*

Uninformed search strategies

- Breadth-first search
- Uniform cost search
- Depth-first search
  - *Depth-limited search*
  - *Iterative deepening search*
- Bidirectional search

Breadth-first search

- The root node is expanded first (FIFO)
- All the nodes generated by the root node are then expanded
- All the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.

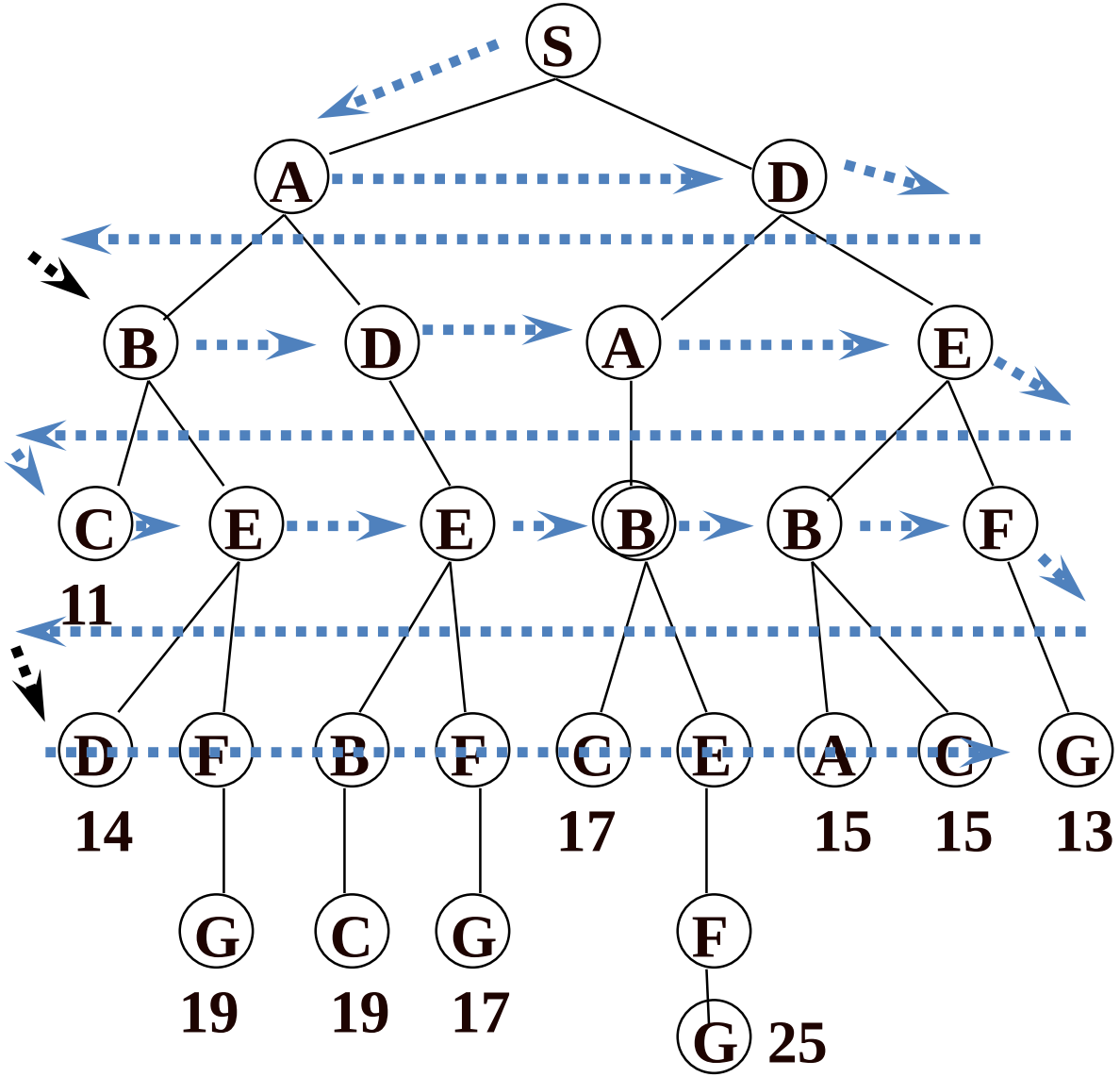


```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Figure 3.11 Breadth-first search on a graph.

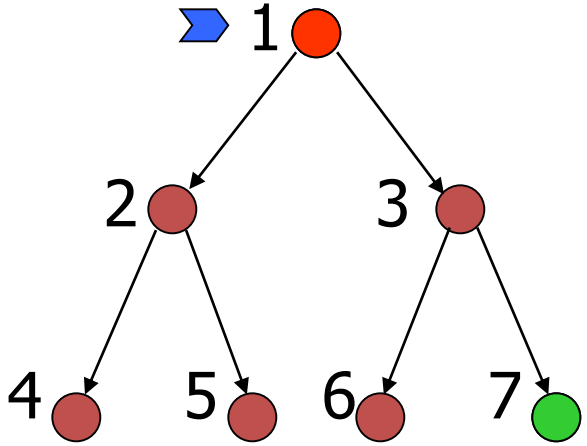
- Initialize a node with the initial state.
- Check if the initial state is already the goal state. If so, return the solution.
- Use a FIFO (first-in-first-out) queue (frontier) to store nodes to be explored.
- Start a loop:
    - If the frontier is empty, return failure (no solution found).
    - Pop the shallowest node (the first in the queue).
    - Add the node’s state to the explored set.
    - For each possible action from the current state, generate a child node.
    - If the child state hasn't been explored and is not in the frontier, check if it is the goal state. If it is, return the solution.
    - Otherwise, add the child to the frontier to explore further.

Breadth-first search



Breadth-First Strategy

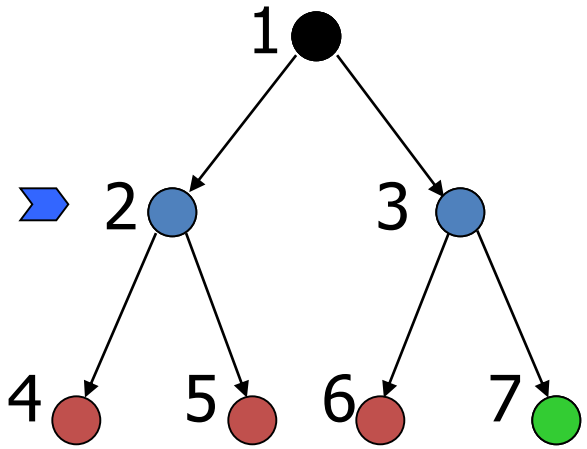
New nodes are inserted at the end of FRINGE



Fringe= (1)

Breadth-First Strategy

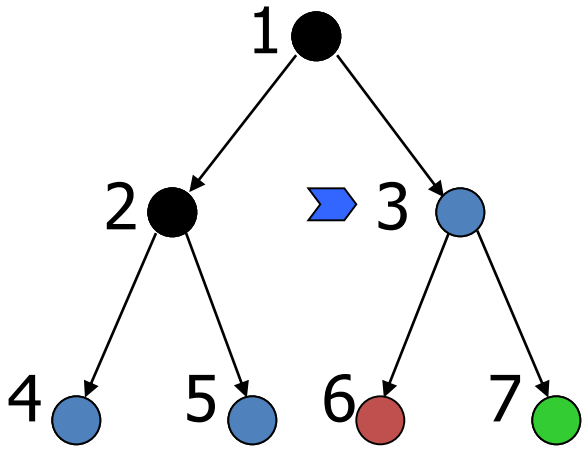
New nodes are inserted at the end of FRINGE



Fringe = (2, 3)

Breadth-First Strategy

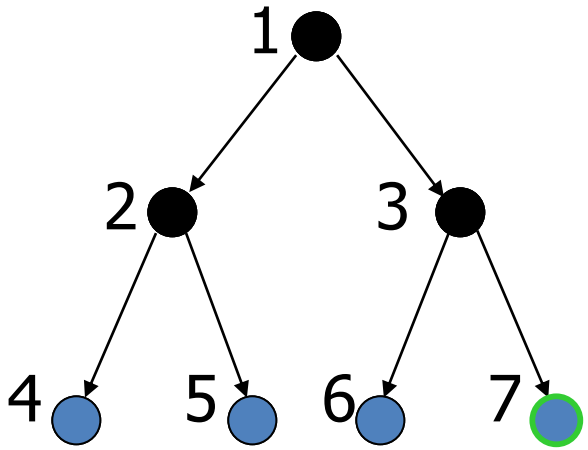
New nodes are inserted at the end of FRINGE



Fringe = (3, 4, 5)

Breadth-First Strategy

New nodes are inserted at the end of FRINGE



Fringe = (4, 5, 6, 7)

Breadth-first search (Analysis)

- Breadth-first search
  - *Complete – if the shallowest goal node is at some finite depth d, breadth-first search will eventually find it after expanding all shallower nodes*
  - *Optimal, if step cost is 1*

  The disadvantage
  - *if the branching factor of a node is large,*
    - the ***space complexity*** and the ***time complexity*** are enormous

Properties of breadth-first search

- Complete? Yes (if b is finite)

- Time? $1+b+b^2+b^3+\ldots+b^d = O(b^d)$ (Worst Case)

When goal test is applied when selected for expansion than when generated then it is - $O(b^{d+1})$

- Explored Set: $O(b^{d-1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)

- Optimal? Yes (if cost = 1 per step)

- Space is the bigger problem (more than time)

Breadth-first search (Analysis)

- assuming 1000 nodes can be processed per second, each with 1000 bytes of storage

Depth	Nodes	Time	Memory
2	1100	.11 seconds	1 megabyte
4	111,100	11 seconds	106 megabytes
6	10^7	19 minutes	10 gigabytes
8	10^9	31 hours	1 terabytes
10	10^{11}	129 days	101 terabytes
12	10^{13}	35 years	10 petabytes
14	10^{15}	3,523 years	1 exabyte

Figure 3.11 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 10,000 nodes/second; 1000 bytes/node.

Disadvantages of breadth-first search

- The memory requirements are a bigger problem for breadth9rst search than is the execution time.
- Exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

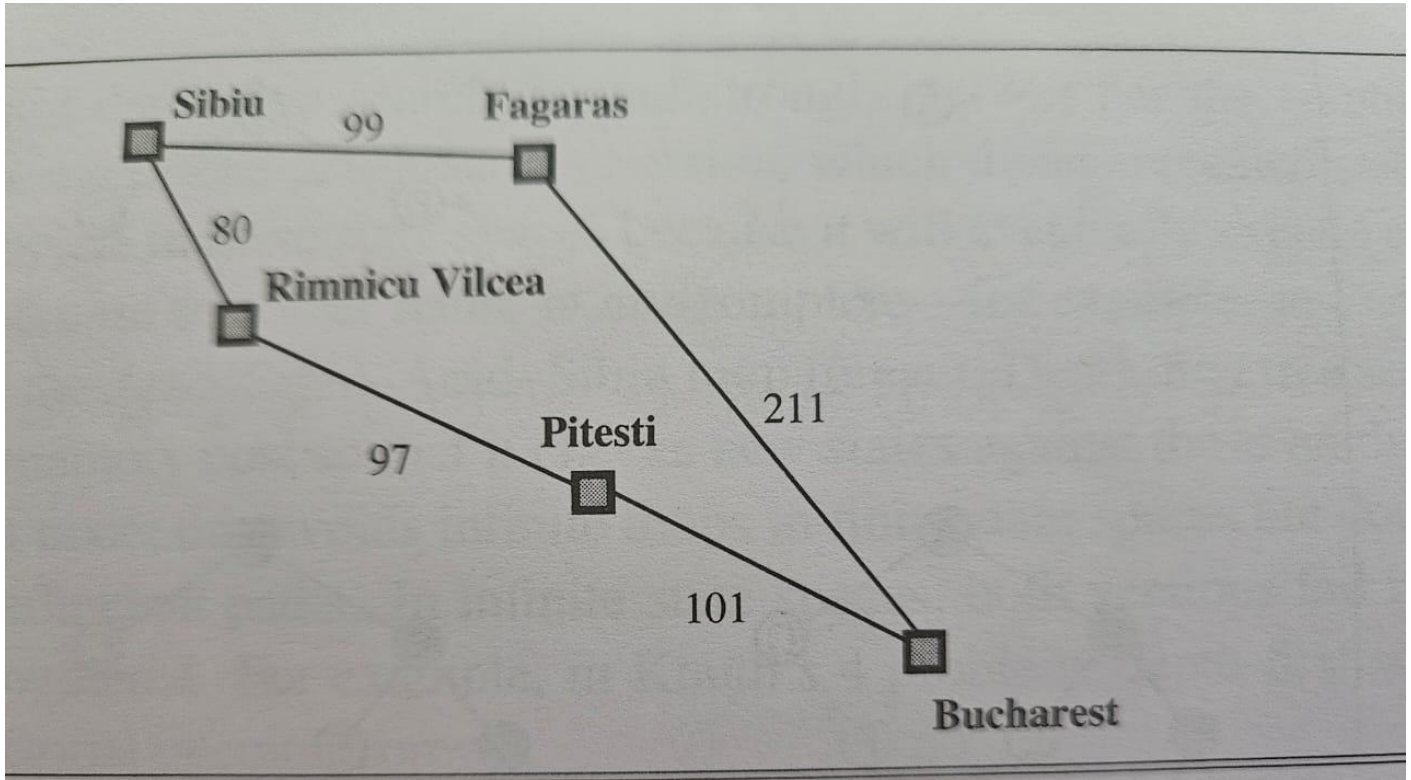
Uniform-Cost Search

- Like breadth-first search
- Expand node with lowest path cost.

Properties:

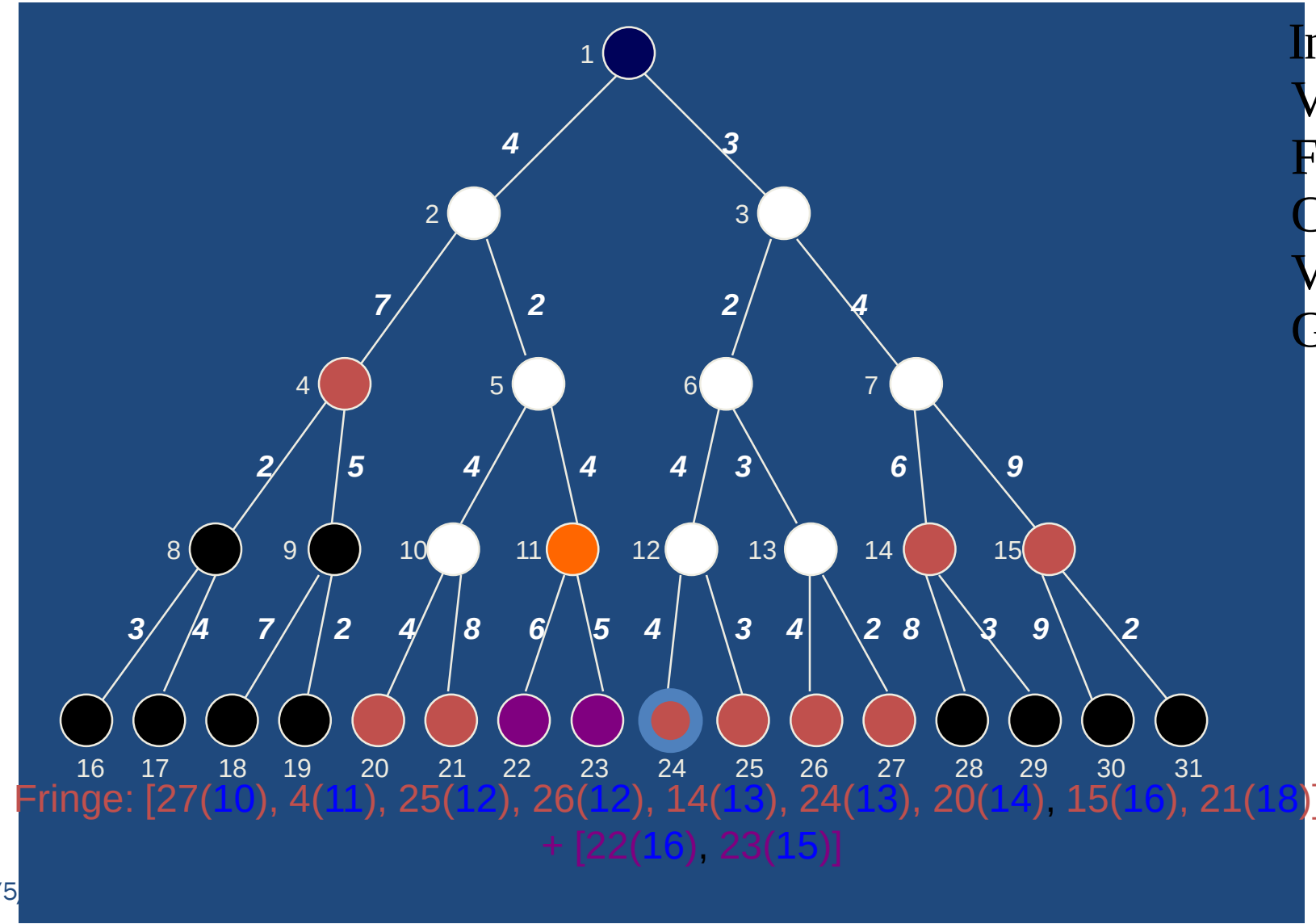
- Complete (if b and d are finite)
- Optimal (if path cost increases with depth)
- Cost is $O(b^{c^*/e})$
- Could be worse than breadth first search

Search



Part of the Romania state space, selected to illustrate uniform-cost

Uniform-Cost Snapshot

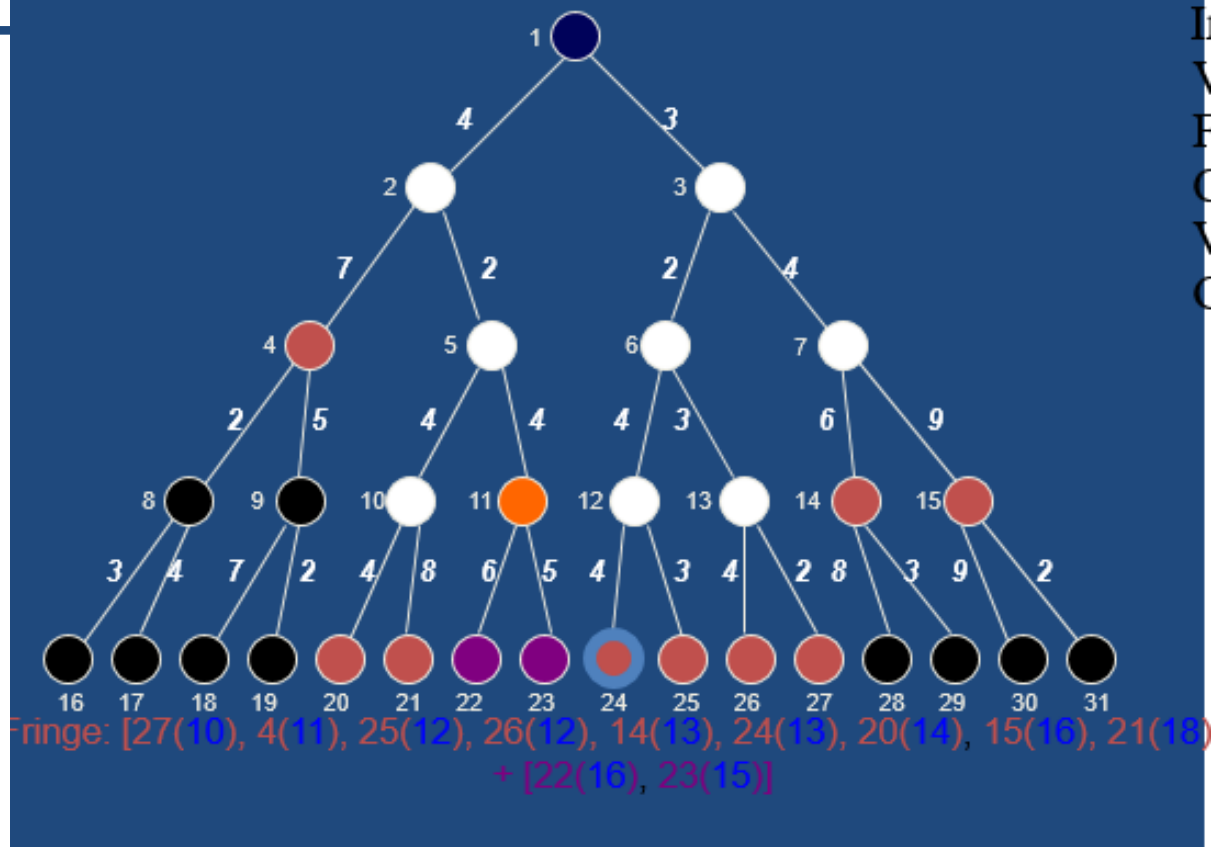


Initial
Visited
Fringe
Current
Visible
Goal

Edge Cost 9

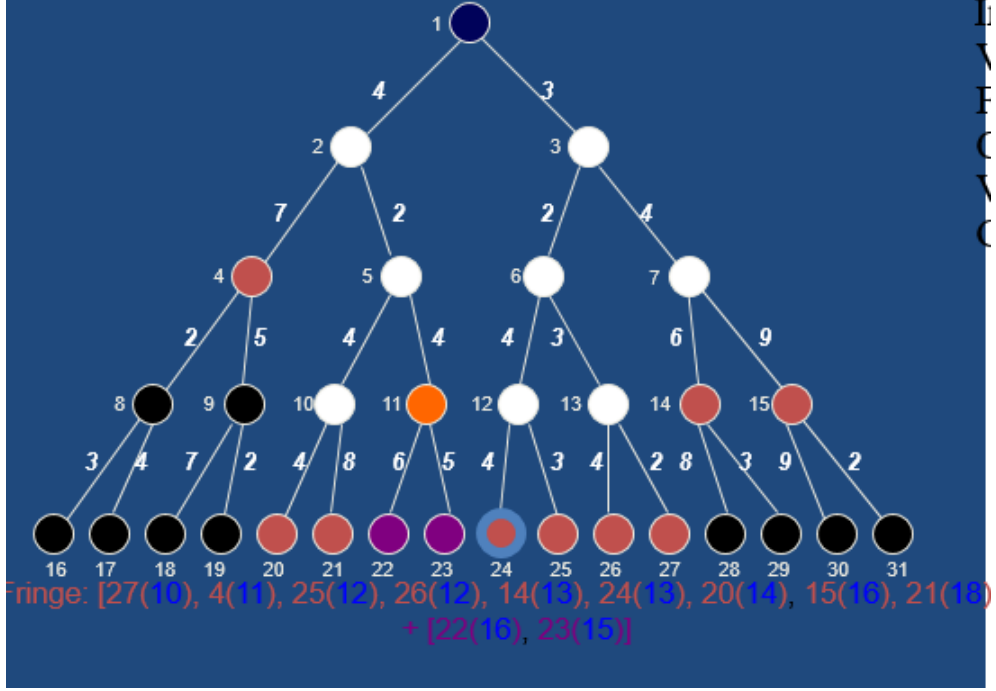
Fringe: [27(10), 4(11), 25(12), 26(12), 14(13), 24(13), 20(14), 15(16), 21(18)]
+ [22(16), 23(15)]

Uniform Cost Fringe T



1. [1(0)]
2. [3(3), 2(4)]
3. [2(4), 6(5), 7(7)]
4. [6(5), 5(6), 7(7), 4(11)]
5. [5(6), 7(7), 13(8), 12(9), 4(11)]
6. [7(7), 13(8), 12(9), 10(10), 11(10), 4(11)]
7. [13(8), 12(9), 10(10), 11(10), 4(11), 14(13), 15(16)]
8. [12(9), 10(10), 11(10), 27(10), 4(11), 26(12), 14(13), 15(16)]

Assumption: New nodes with the same cost as existing nodes are added after the existing node.



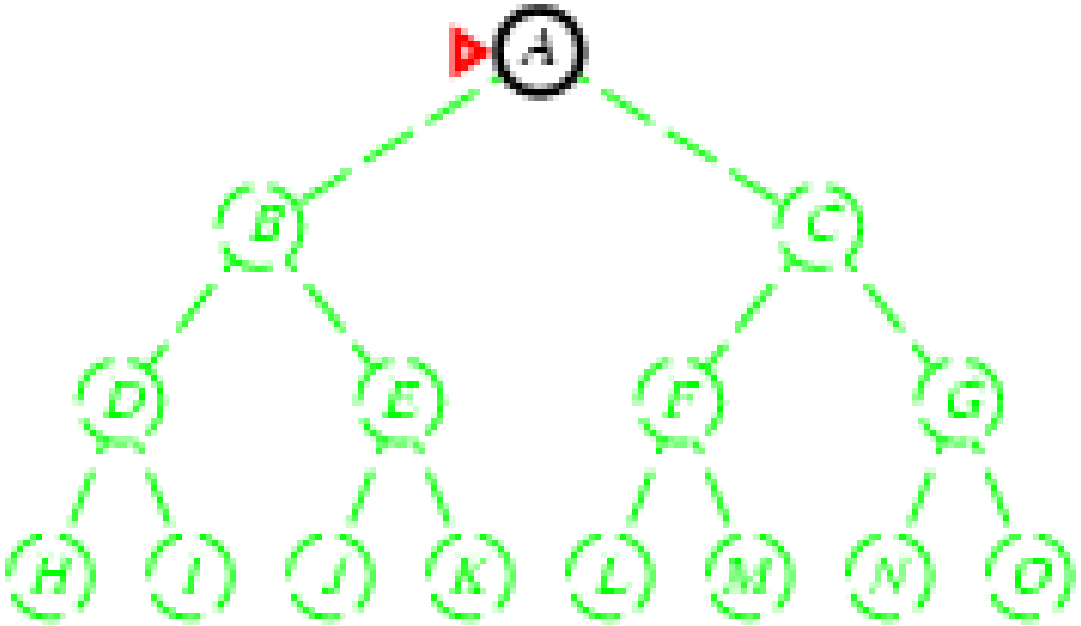
1. [10(10), 11(10), 27(10), 4(11), 26(12), 25(12), 14(13), 24(13), 15(16)]
2. [11(10), 27(10), 4(11), 25(12), 26(12), 14(13), 24(13), 20(14), 15(16), 21(18)]
3. [27(10), 4(11), 25(12), 26(12), 14(13), 24(13), 20(14), 23(15), 15(16), 22(16), 21(18)]
4. [4(11), 25(12), 26(12), 14(13), 24(13), 20(14), 23(15), 15(16), 22(16), 21(18)]
5. [25(12), 26(12), 14(13), 24(13),8(13), 20(14), 23(15), 15(16), 22(16), 9(16), 21(18)]
6. [26(12), 14(13), 24(13),8(13), 20(14), 23(15), 15(16), 22(16), 9(16), 21(18)]
7. [14(13), 24(13),8(13), 20(14), 23(15), 15(16), 22(16), 9(16), 21(18)]
8. [24(13),8(13), 20(14), 23(15), 15(16), 22(16), 9(16), 29(16),21(18), 28(21)]
9. *Goal reached!*

Depth-first search

- Always expands one of the nodes at the ***deepest*** level of the tree
- Only when the search hits a dead end
  - *goes back and expands nodes at shallower levels*
  - *Dead end → leaf nodes but not the goal*
- Backtracking search
  - *only one successor is generated on expansion*
  - *rather than all successors*
  - *fewer memory*

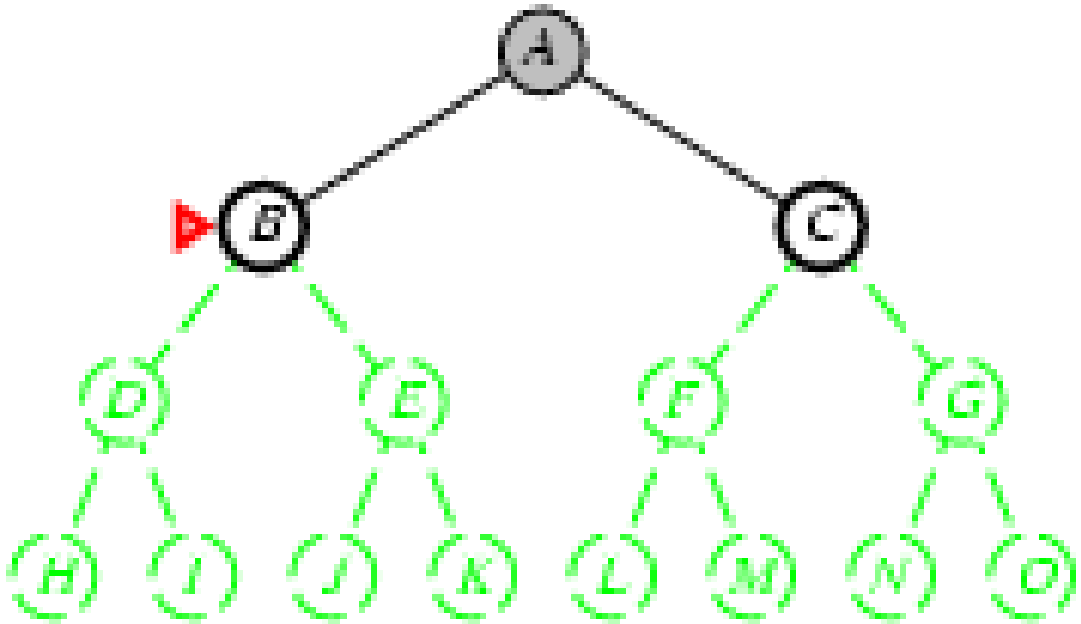
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



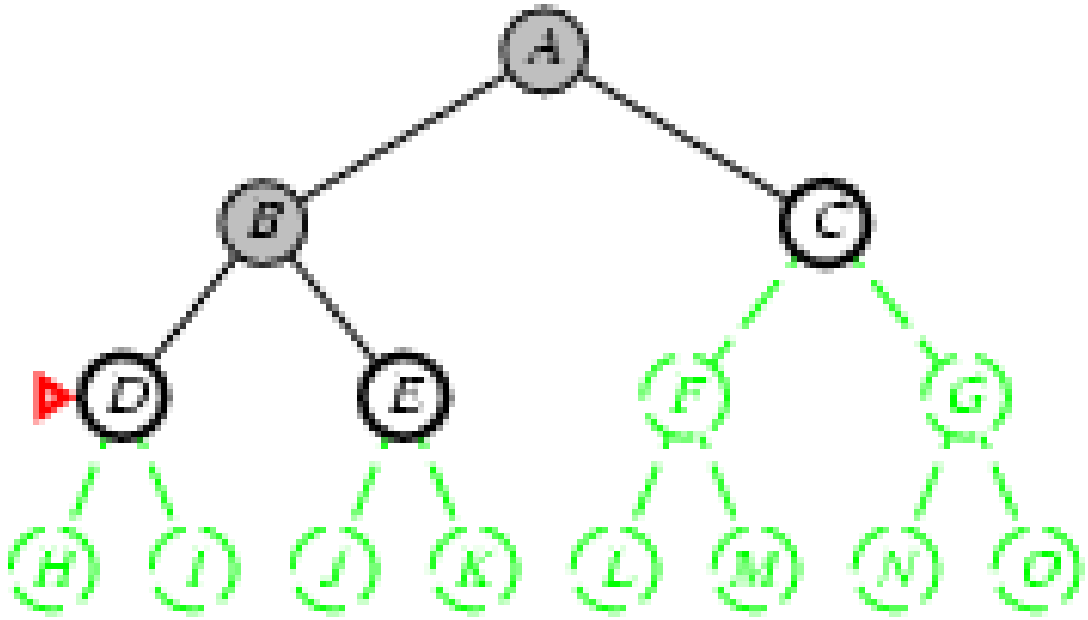
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



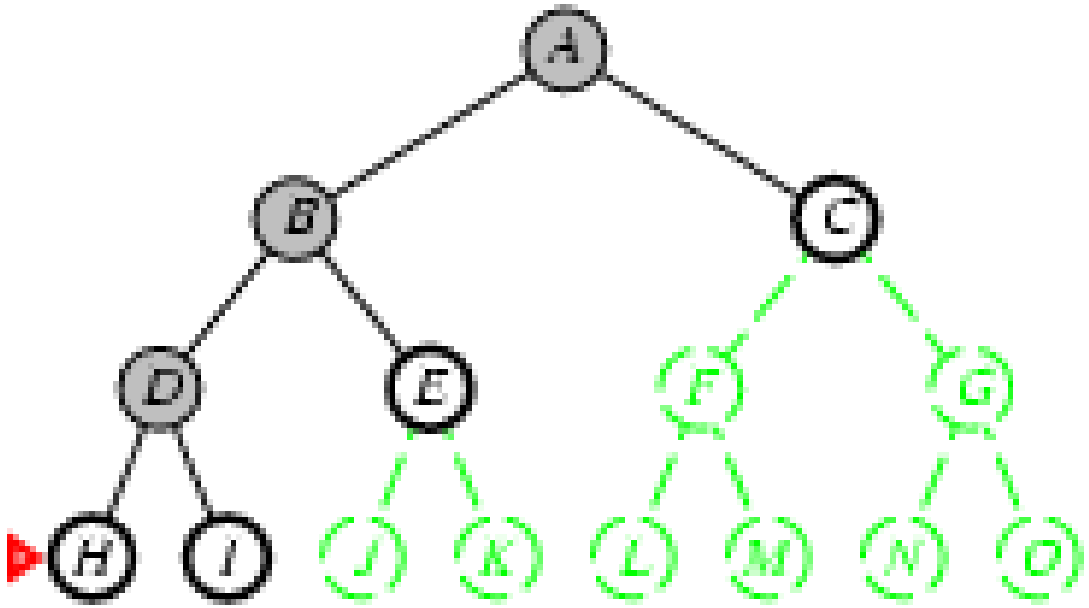
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



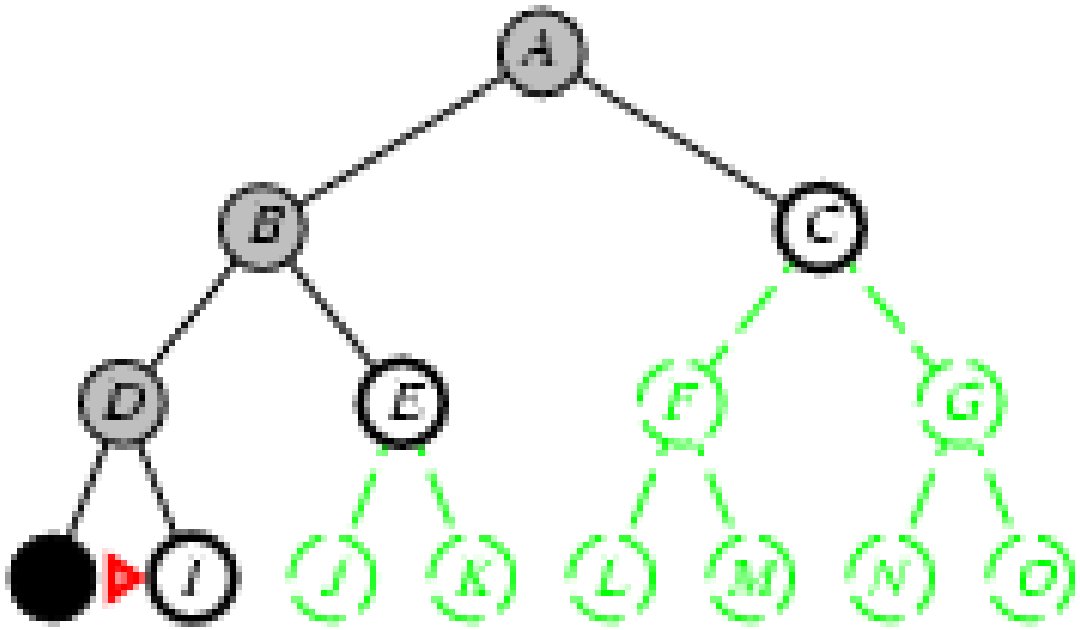
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



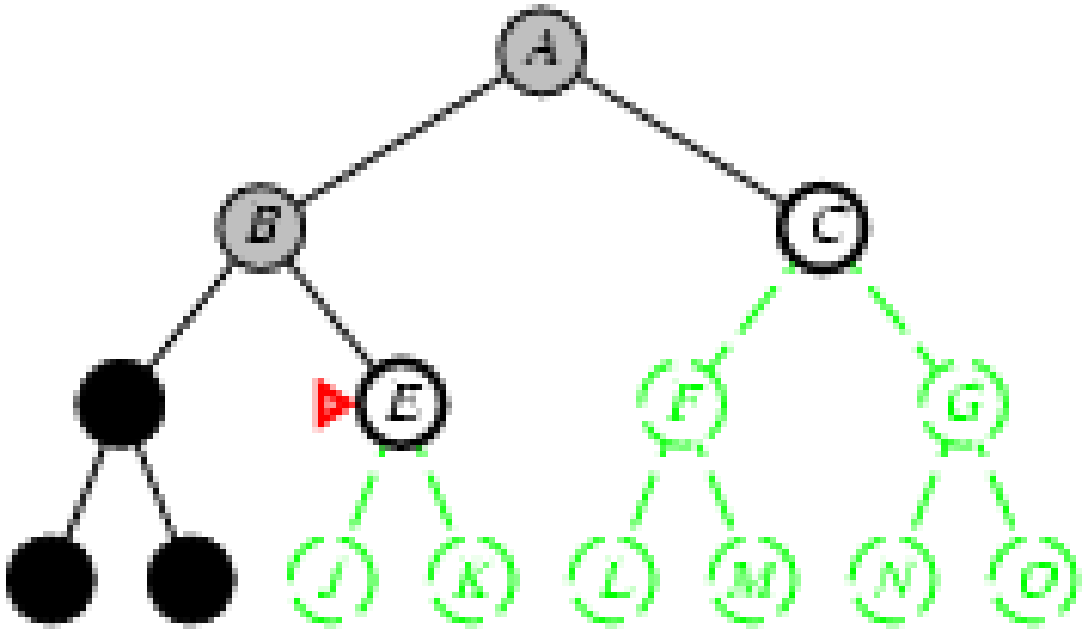
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



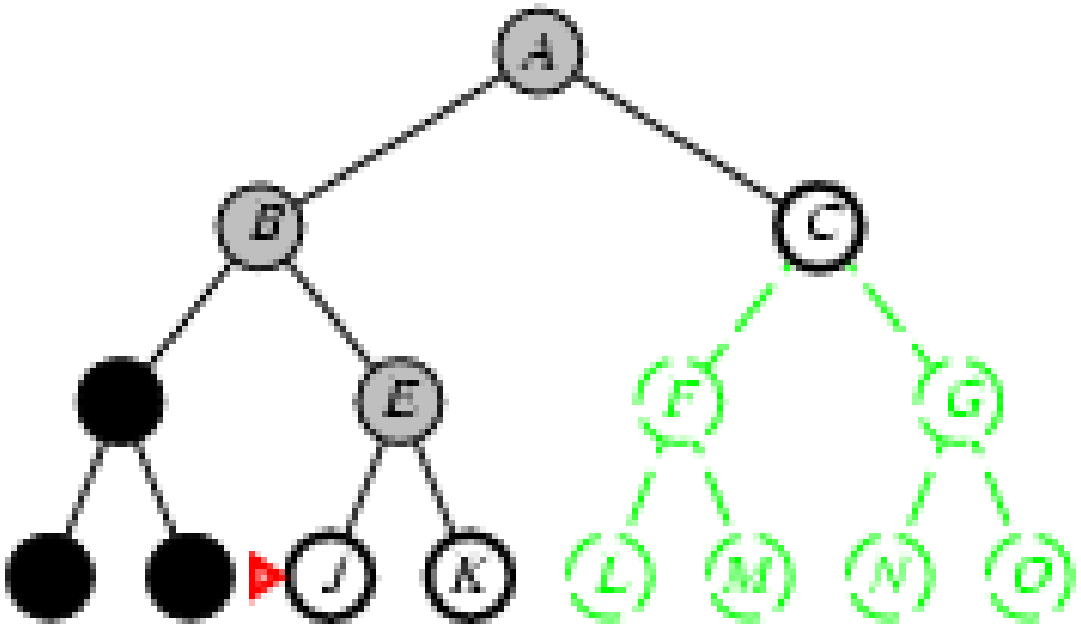
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



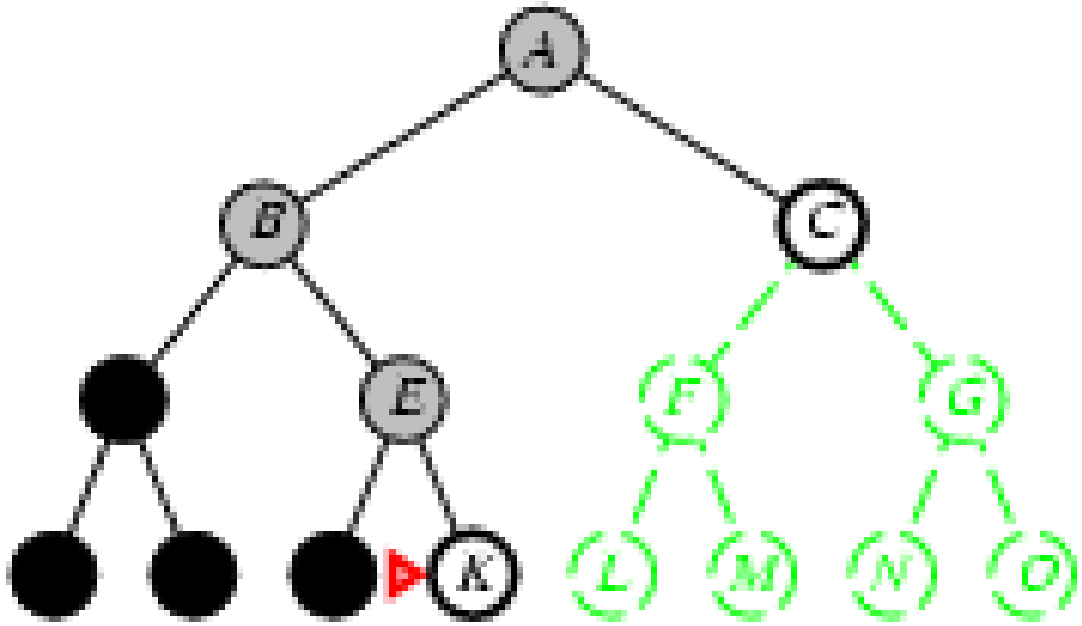
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



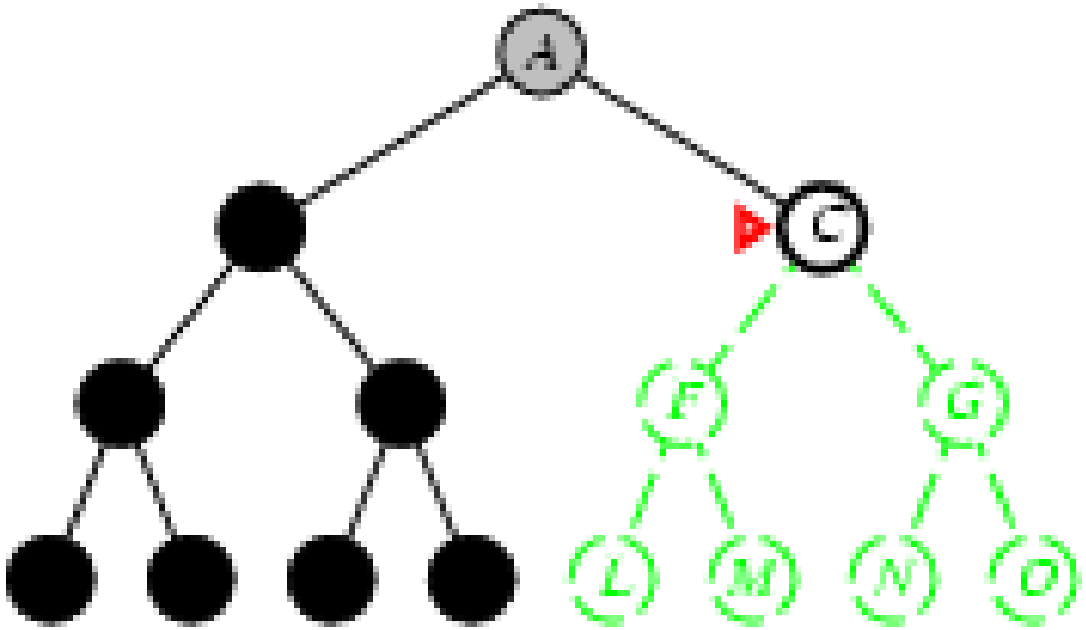
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



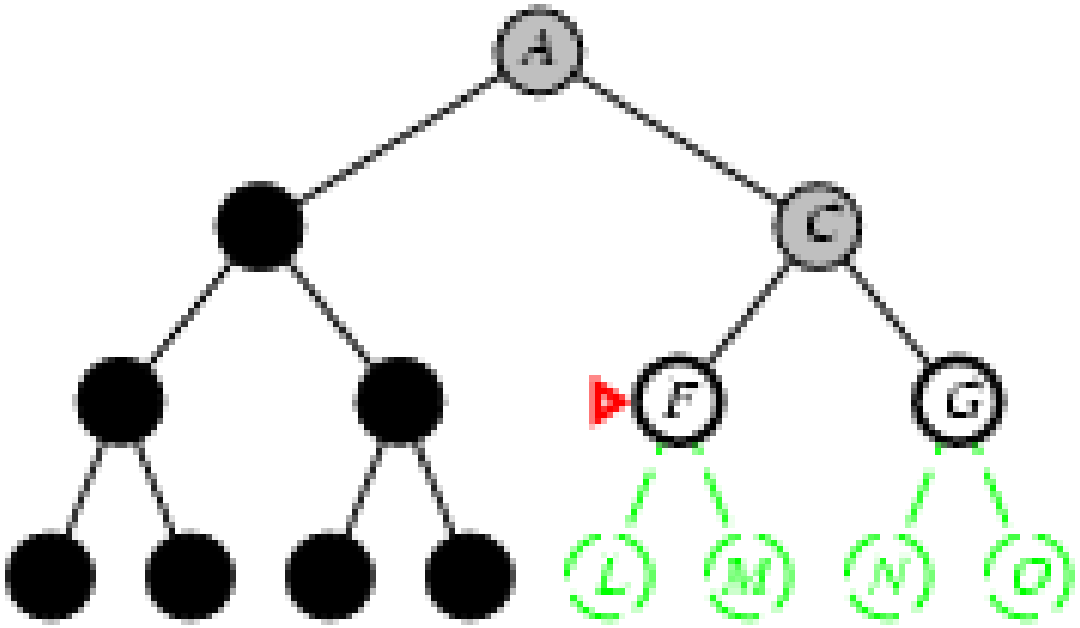
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



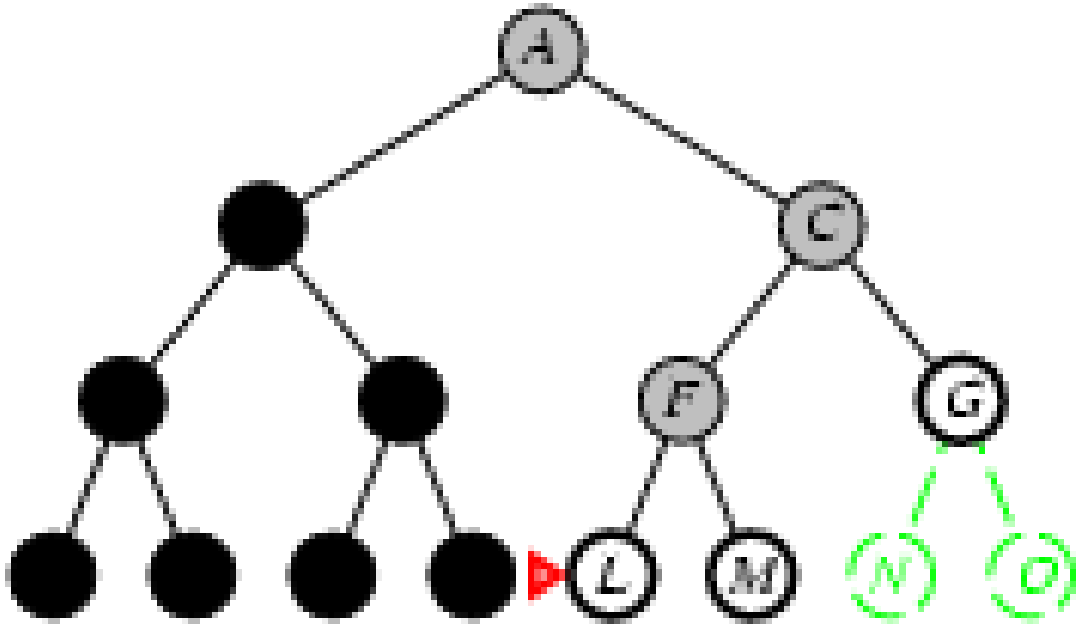
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



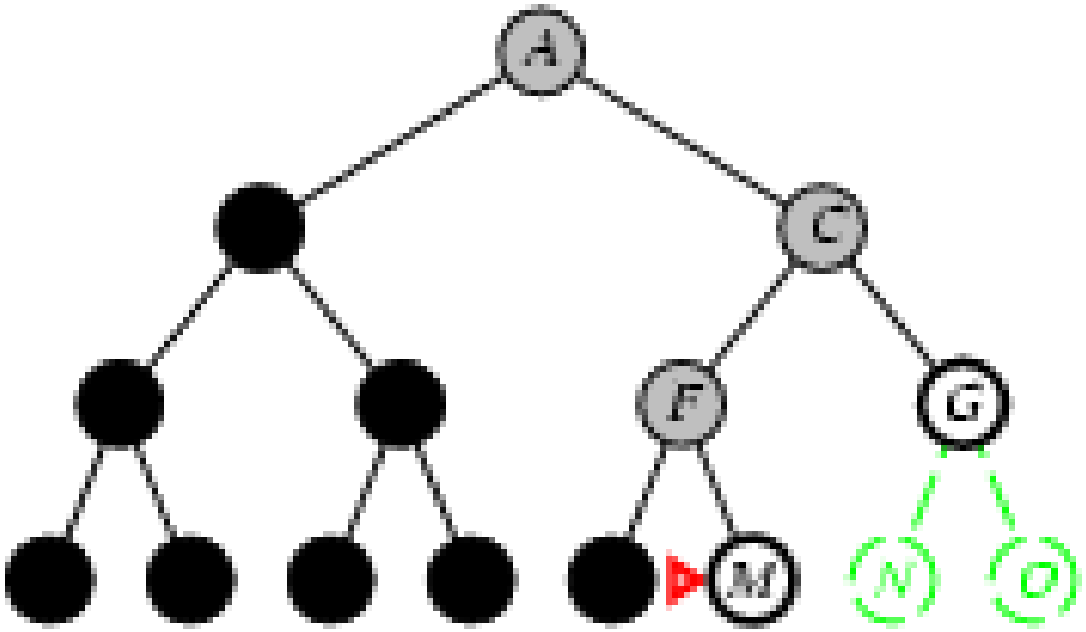
Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*

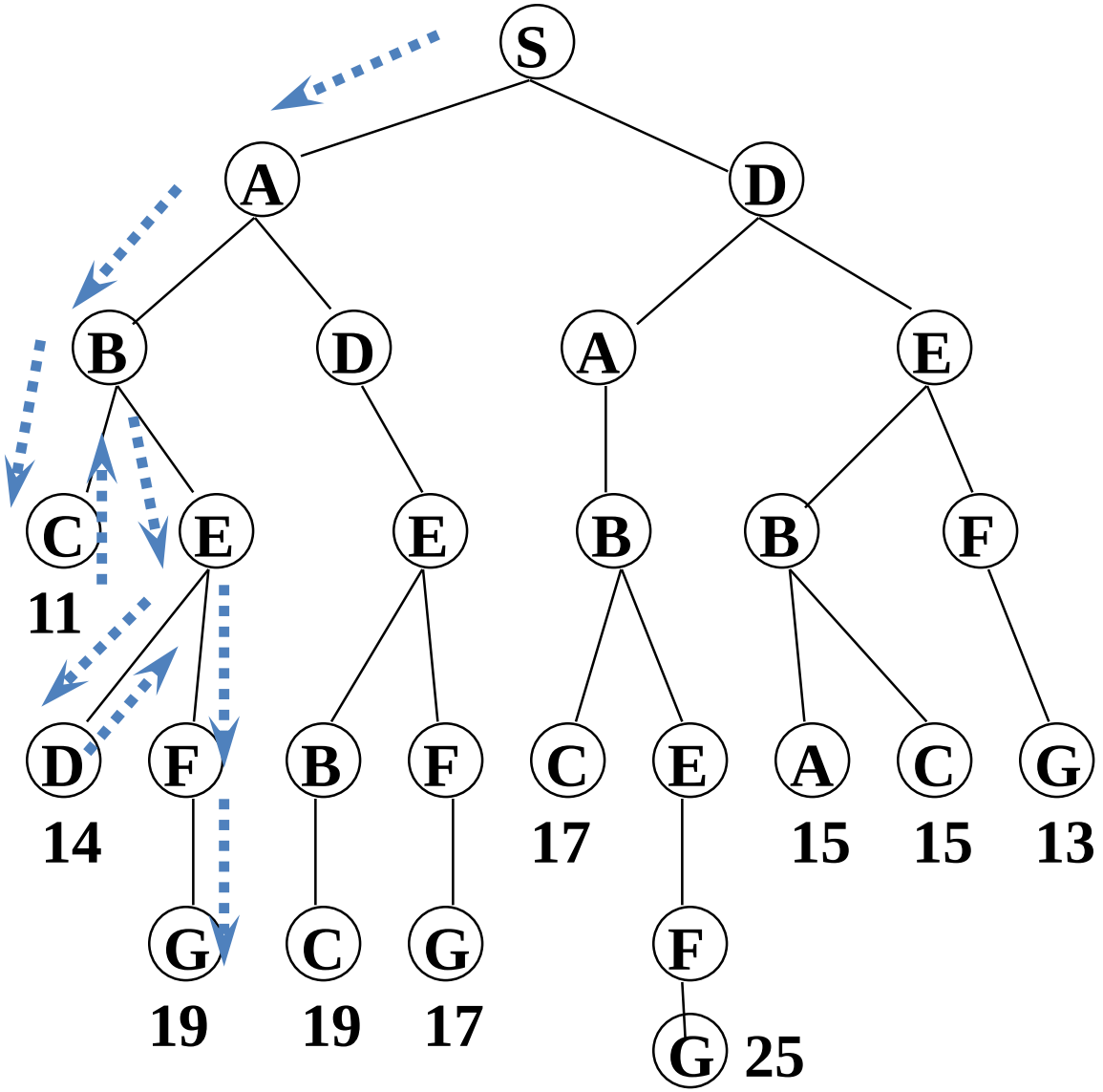


Depth-first search

- Expand deepest unexpanded node
- Implementation:
  - *fringe = LIFO queue, i.e., put successors at front*



Depth-first search



Depth-first search (Analysis)

- Not complete
  - *because a path may be infinite or looping*
  - *then the path will never fail and go back try another option*
- Not optimal
  - *it doesn't guarantee the best solution*
- It overcomes
  - *the time and space complexities*

Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops

  → complete in finite spaces

- Time? $O(b^m)$: terrible if m is much larger than d
  - *but if solutions are dense, may be much faster than breadth-first*
  - *b is the branching factor, which is the maximum number of children each node can have.*
  - *m is the maximum depth (or height) of the search tree.*

- Space? *O(bm),* i.e., linear space!

- Optimal? No

Depth-Limited

- Like depth-first search but with depth limit L.

Properties:

- Incomplete (if L < d)
- non optimal (if L > d)
- Time complexity is $O(b^L)$
- Space complexity is O(bL)

- Completeness: DLS is incomplete if the depth limit LL is less than the actual depth of the solution dd. This means it might miss solutions that exist below the limit.
- Optimality: DLS is non-optimal if the depth limit LL is greater than dd. In such cases, it may find a solution, but it might not be the most optimal (i.e., shallowest) solution.
- Time Complexity: The time complexity of DLS is $O(b^L)$ where b is the branching factor, and L is the depth limit. This represents the maximum number of nodes that could be visited, as the search explores down to the limit L but no further.
- Space Complexity: The space complexity is also $O(bL)$, as the algorithm only needs to store up to L levels of the search tree in the recursion stack at any time. This makes it more memory-efficient than traditional DFS in deep trees.

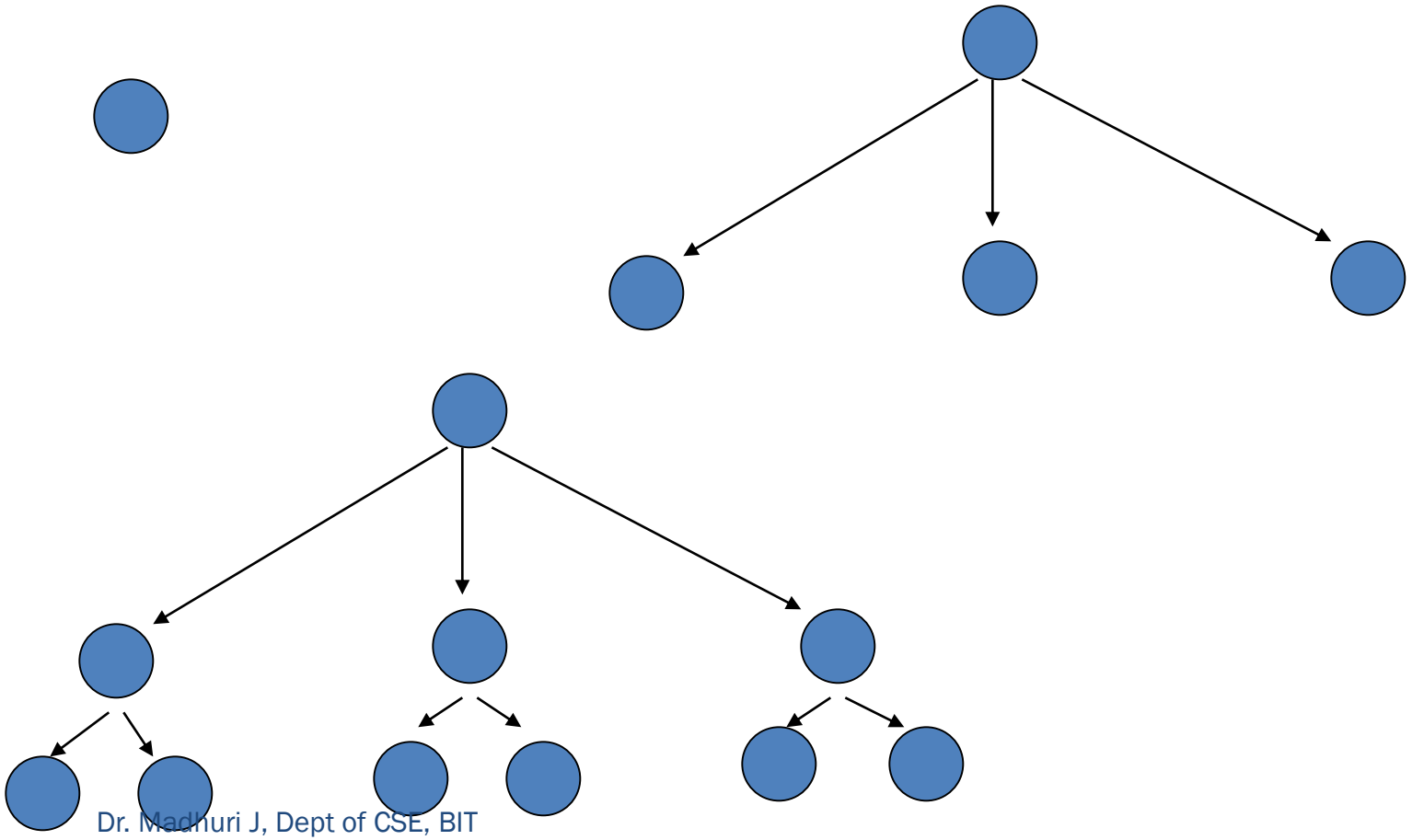
Iterative Deepening

- A combination of depth and breadth-first search.
- Gradually increases the limit **L**
- Commonly used in scenarios where the depth of the solution is unknown, combining the memory efficiency of DFS with the completeness and optimality of BFS.

Properties:

- Complete (if b and d are finite)
- Optimal if path cost increases with depth
- Time complexity is $O(\mathbf{b}^{\mathbf{d}})$

Search



Bidirectional Search

- Run two searches – one from the initial state and one backward from the goal.
- We stop when the two searches meet.

Motivation:

- Time complexity: $(b^{d/2} + b^{d/2}) < b^{d}$

Searching backwards not easy.

Problem Solving By Searching

- Introduction
- Solutions and Performance
- Uninformed Search Strategies
- ***Avoiding Repeated States/Looping***
- Partial Information
- Summary

Repeated States/Looping

- Avoiding redundant state expansions to enhance efficiency. In a state space structured like a tree, where each state can only be reached by a single path, redundancy is naturally limited.
- However, in cases where a state space has cycles or allows multiple paths to the same state (as seen in some problems like route-finding or puzzles), failing to track visited states can lead to an overwhelming number of expansions.
- The solution often involves employing a "closed list" in an approach called GRAPH-SEARCH, where every expanded state is stored and checked before further expansion. This method greatly reduces the number of repeated state explorations
- However, maintaining a closed list increases memory demands, especially for depth-first search and iterative deepening search.
- This space-time tradeoff underscores a core challenge in search algorithms: balancing the memory required to store state histories against the computational time saved by avoiding redundant searches.

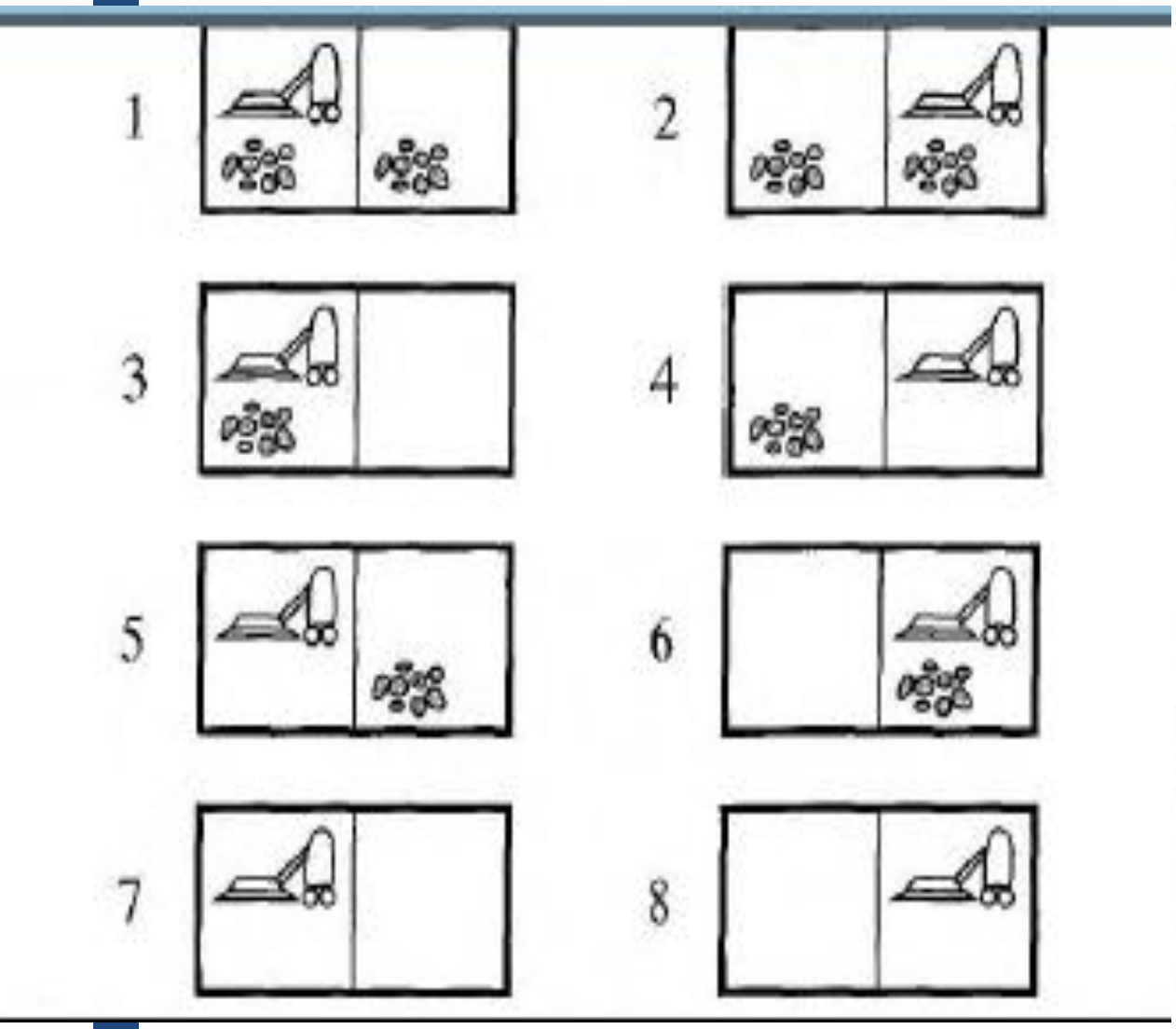
Avoiding Looping & Repeated States

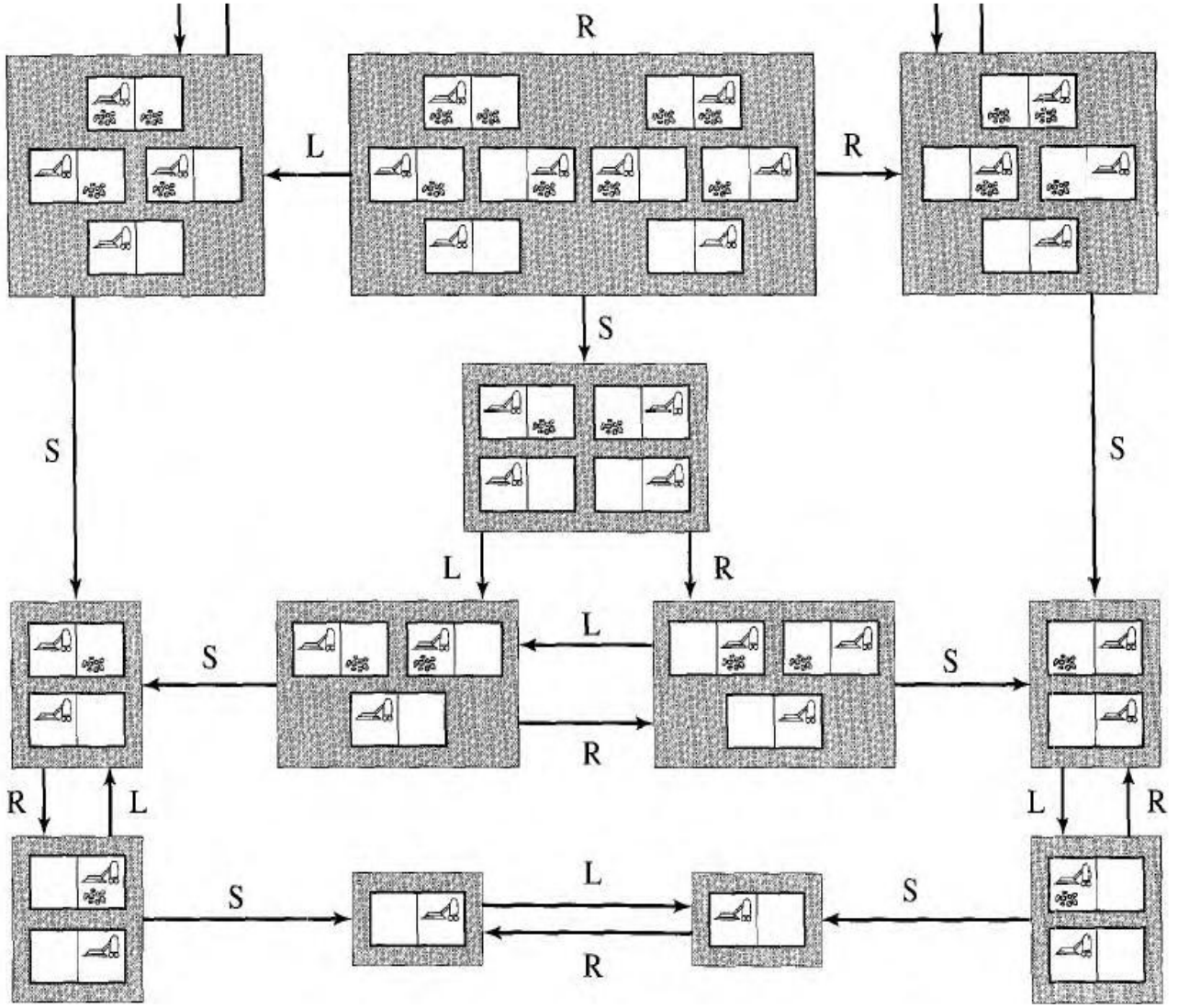
- Use a list of expanded states; non-expanded states (*open* and *close list*)
- Use domain specific knowledge
- Use sophisticated data structures to find already visited states more quickly.
- Checking for repeated states can be quite expensive and slow down the search alg.

SEARCHING WITH PARTIAL INFORMATION

- **Sensorless problems** (also called **conformant problems):** If the agent has no sensors at all, then (as far as it knows) it could be in one of several possible initial states, and each action might therefore lead to one of several possible successor states.
- **Contingency problems:** If the environment is partially observable or if actions are uncertain, then the agent's percepts provide ***new*** information after each action. Each possible percept defines a contingency that must be planned for. A problem is called **adversarial** if the uncertainty is caused by the actions of another agent.
- **Exploration problems**: When the states and actions of the environment are unknown, the agent must act to discover them. Exploration problems can be viewed as an extreme case of contingency problems.

- A *sensorless* vacuum agent operates without any sensory input, relying solely on its knowledge of action effects.
- To solve sensorless problems, we search in the space of belief states rather than physical states. The initial state is a belief state, and each action maps from a belief state to another belief state.
- **Initial Belief State**: The vacuum agent knows it could start in any state within a set of possible states (e.g., {1,2,3,4,5,6,7,8}).
- **Action-Based State Calculation:** Despite lacking sensors, the agent can determine which states it could be in after certain actions, e.g., the action sequence [Right, Suck, Left, Suck] always leads to the goal state 7, regardless of the initial state.


1
2
3
4
5
6
7
8


R
L
R
S
S
S
L
R
L
S
L
S
R
R
L
R
L
S
L
S
R

- **Belief State Space Complexity**: For a space with S physical states, the belief state space contains 2S2^S2S possible belief states. In the sensorless vacuum example, there are only 12 reachable belief states out of a possible 256.
- **Deterministic vs. Nondeterministic Environments**: In a deterministic environment, actions lead to predictable belief state changes. In a nondeterministic environment (e.g., Murphy's Law), where actions may have multiple outcomes, belief states become unions of possible outcomes.
- **Reasoning in Partially Observable Worlds**: Without full observability, the agent reasons over sets of possible states (belief states), unlike fully observable cases where each belief state maps to a single physical state.

Contingency problems

- In a contingency problem, an agent must adapt its actions based on new information received through its sensors after each step. This leads to decision-making paths that can change depending on the environment.
- **Example - Murphy's Law World**: An agent in Murphy's Law world, with limited sensory information (like a position sensor and a local dirt sensor), faces unpredictability. For instance, it may take an action like "Suck" to clean dirt, but due to Murphy's Law, dirt might reappear or other unintended changes might occur, requiring the agent to adjust its actions dynamically.
- A fixed action sequence may fail because of contingencies (e.g., unexpected dirt reappearing). Instead, agents need conditional plans like [Suck, Right, if [R, Dirty] then Suck], allowing them to adjust actions based on current conditions.
- Contingency problems are common in real-world scenarios where precise predictions are impossible, such as in driving or walking, where constant adjustment based on new sensory input is necessary.

Contingency problems

- **Sequential Solutions in Fully Observable Environments:** In fully observable environments (where all state information is visible), some problems may allow sequential solutions without conditional branching, simplifying the solution.
- **Interleaving Search and Execution**: In complex environments, an agent might begin executing actions even without a fully guaranteed plan. This interleaving approach helps by allowing the agent to respond to arising contingencies in real-time, which is also applicable in areas like exploration and game playing.